



acm International Collegiate
Programming Contest



event
sponsor

基础算法

Ciallo~(∠・ω<)〰☆

目 录

基础算法	3
前缀和&差分	3
一维前缀和	3
二维前缀和	3
一维差分	5
二维差分	6
二分	7
二分查找	7
二分答案	8
实数域二分	10
分数规划	10
子段和问题	12
三分	13
双指针	14
排序	15
sort	15
快速排序	16
第k小的数	17
归并排序	17
逆序对	18
分治	19
二进制	21
lowbit	21
状态压缩	22
枚举子掩码	25
位运算	25
^ 异或	26
& 与	27
>> 移位	27
离散化	27
搜索	28
DFS	28
剪枝	31

迭代加深	34
BFS	35
双端队列BFS	36
优先队列BFS	39
双向搜索	41
A*	44
IDA*	46
Dancing Links	46
精确覆盖	46
重复覆盖	51
倍增	51
ST表	51
贪心	55
区间问题	55
区间选点	55
区间合并	57
不相交区间	58
区间覆盖	59
区间分组	60
区间包含	61
绝对值不等式	63
排序不等式	65
后悔解法	67
根号分治	68
离线算法	70
莫队	70
普通莫队	70
带修莫队	72
回滚莫队	74
只增回滚莫队	74
只删回滚莫队	78
随机化	80

基础算法

前缀和&差分

在头文件中也包含了一维前缀和/差分函数

```
int a[6] = {1,2,3,4,5,6},s[6];
partial_sum(a,a+6,s); //s = {1,3,6,10,15,21}
adjacent_difference(a,a+6,s); //s = {1,1,1,1,1,1}
```

一维前缀和

$pre[i] = a[1] + a[2] + \dots + a[i] = pre[i-1] + a[i]$

前缀和数组一般下标从1开始，方便写

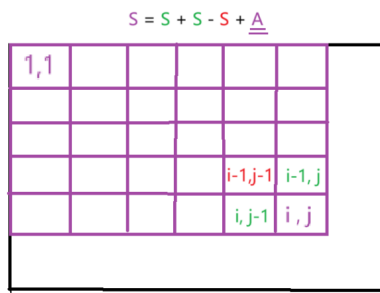
```
1 //求第a个数到第b个数的和
2 #include<iostream>
3 using namespace std;
4 int arr[1000006]; int pre[1000006];
5 int main() {
6     int k; cin >> k;
7     for (int i = 1; i <= k; i++){
8         scanf("%d", &arr[i]);
9         pre[i] += pre[i - 1] + arr[i];
10    }
11    int a, b; cin >> a >> b;
12    cout << pre[b] - pre[a - 1] << endl;
13    return 0;
14 }
```

```
1 //异或前缀和
2 for(int i = 1; i <= n; i++){
3     s[i] = s[i-1]^a[i];
4 }
5 ans = s[r]^s[l-1];
```

二维前缀和

子矩阵的和：

```
1 //每个点的S的求法,可以直接推
2 s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j];
```



```
1 //某一块S的求法: s[x1][y1]~s[x2][y2]
2 s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1];
```

			x1,y1			
				x2,y2		

```
1 #include<iostream>
2 using namespace std;
3 const int N = 10006;
4 int arr[N][N]; int s[N][N];
5 int n, m;
6
7 int query(int x1,int y1,int x2,int y2){
8     return s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1];
9 }
10
11 int main() {
12     cin >> n >> m;
13     for (int i = 1; i <= n; i++)
14         for (int j = 1; j <= m; j++)
15             scanf("%d", &arr[i][j]);
16     for (int i = 1; i <= n; i++) //利用前缀和求每一点的S
17         for (int j = 1; j <= m; j++)
18             s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + arr[i][j];
19 }
```

```

20     int x1, y1, x2, y2;
21     cin >> x1 >> y1 >> x2 >> y2; //求子矩阵x1, y1 ~ x2, y2的S
22     cout << query(x1,y1,x2,y2) << endl;
23     return 0;
24 }

```

一维差分

$\text{dif}[1] + \text{dif}[2] + \dots + \text{dif}[i] = a[i];$

往往与前缀和联系

$\text{dif}[i] = \text{arr}[i] - \text{arr}[i-1];$

```

1 //https://www.luogu.com.cn/problem/P2367
2 #include <iostream>
3 using namespace std;
4 int arr[5000006], dif[5000006];
5 int main() {
6     int n, p; cin >> n >> p;
7     for (int i = 1; i <= n; i++){
8         scanf("%d", &arr[i]); //输入数据较多时建议scanf/printf 比cin/cout快,
9         dif[i] = arr[i] - arr[i - 1];
10    }
11    while (p--){
12        int x, y, z;
13        scanf("%d%d%d", &x, &y, &z); //区间[x,y]加x
14        dif[x] += z;
15        dif[y + 1] -= z;
16    }
17    int low = 1e9;
18    for (int i = 1; i <= n; i++){
19        arr[i] = arr[i - 1] + dif[i];
20        low = low < arr[i] ? low : arr[i];
21    }
22    cout << low << endl;
23    return 0;
24 }

```

```

1 //增减序列: https://www.acwing.com/problem/content/102/
2 //给定数组a[n],每次可选一个区间[l,r]内的数都+1或-1, 求使所有数都一样的最小操作次数和最终序列
  有多少种
3 #include <iostream>
4 #include <algorithm>
5 using namespace std;
6 const int N = 100005;
7 using ll = long long;
8 int n;
9
10 ll a[N], dif[N];

```

```

11
12 int main(){
13     cin >> n;
14     for(int i = 1; i <= n; i++){
15         cin >> a[i];
16         dif[i] = a[i] - a[i-1];
17     }
18
19     ll pos = 0, neg = 0;
20
21     //如果这个数列的值都是一样的,最后我们的差分数组一定是b[1]=一个常数,b[2]~b[n]都等于0
22     for(int i = 2; i <= n; i++){
23         if(dif[i] > 0) pos += dif[i]; //pos表示sum(正数->0)
24         if(dif[i] < 0) neg += -dif[i]; //neg表示sum(负数->0)
25     } //正负两个数可以相消二者取最小min,最后差分就只剩同符号的数abs(pos-neg)
26     cout << min(pos, neg) + abs(pos - neg) << endl;
27     cout << abs(pos - neg) + 1;
28     return 0;
29 }

```

二维差分

```

1 //https://www.acwing.com/problem/content/800/
2 //给定n*m的矩阵和q次插入操作,每次操作将子矩阵(x1,y1,x2,y2)的值加上c,求q次操作后的矩阵
3 #include <iostream>
4 using namespace std;
5 int a[1006][1006]; int dif[1006][1006];
6 int n, m, q;
7
8 void insert(int x1, int y1, int x2, int y2, int c) {
9     dif[x1][y1] += c;
10    dif[x2 + 1][y1] -= c;
11    dif[x1][y2 + 1] -= c;
12    dif[x2 + 1][y2 + 1] += c;
13 }
14
15 int main() {
16     cin >> n >> m >> q;
17     for (int i = 1; i <= n; i++){
18         for (int j = 1; j <= m; j++){
19             scanf("%d", &a[i][j]);
20             insert(i, j, i, j, a[i][j]); //初始化插入a[i][j]
21         }
22     }
23
24     while (q--) {

```

```

25     int x1, y1, x2, y2, c; //进行q次插入操作
26     scanf("%d%d%d%d", &x1, &y1, &x2, &y2, &c);
27     insert(x1, y1, x2, y2, c);
28 }
29
30 for (int i = 1; i <= n; i++)//利用前缀和求改变后的数组
31     for (int j = 1; j <= m; j++)
32         dif[i][j] += dif[i-1][j] + dif[i][j-1] - dif[i-1][j-1];
33
34 for (int i = 1; i <= n; i++) {
35     for (int j = 1; j <= m; j++)
36         printf("%d ", dif[i][j]);
37     printf("\n");
38 }
39 }

```

二分

二分查找

目标数组需要为非降序排列

```

1  #include <iostream>
2  using namespace std;
3  int n, x, arr[1000006];
4  bool check1(int mid) {
5      return arr[mid] >= x;
6  }
7  bool check2(int mid) {
8      return arr[mid] <= x;
9  }
10
11 int main() {
12     scanf("%d%d", &n, &x);
13     for (int i = 0; i < n; i++) {
14         scanf("%d", &arr[i]);
15     }
16
17     int l = 0, r = n - 1; //找到第一个>=x值的下标    (与lower_bound略有不同)
18     while (l < r) { //0000x1111型 (x为目标)
19         int mid = l + r >> 1;
20         if (check1(mid)) r = mid;
21         else l = mid + 1;
22     }

```

```

23     if(arr[l] < x) cout << -1 << endl; // 不存在 >= x 的值, 返回的是最后一个元素下标, 而 lb
    返回的是 end()
24     else cout << l << endl;
25
26     l = 0, r = n - 1; // 找到最后一个 <= x 值的下标 (与 upper_bound 不同)
27     while (l < r) { // 1111x0000 型 (x 为目标)
28         int mid = l + r + 1 >> 1;
29         if (check2(mid)) l = mid;
30         else r = mid - 1;
31     }
32     if(arr[l] > x) cout << -1 << endl; // 不存在 <= x 的值, 返回的是第一个元素的下标
33     cout << l << endl;
34
35     return 0;
36 }

```

二分答案

单调区间中, 每次查找去掉不符合条件的一半区间, 直到找到答案 (整数二分) 或者和答案十分接近

```

1 // 模版与二分查找一样
2 // check(mid) 为: ...00001111... 型 (第一个 1 为目标)
3 int l = 0, r = 1e9;
4 while (l < r) {
5     int mid = l + r >> 1;
6     if (check(mid)) r = mid;
7     else l = mid + 1;
8 }
9 cout << l;
10
11
12 // check(mid) 为: ...111110000... 型 (最后一个 1 为目标)
13 int l = 0, r = 1e9;
14 while (l < r) {
15     int mid = l + r + 1 >> 1;
16     if (check(mid)) l = mid;
17     else r = mid - 1;
18 }
19 cout << l;

```

```

1 // 最小化极值: https://codeforces.com/contest/2013/problem/D
2 // 题意: 对于数组 a, 可以任意次选择 i (1~n-1), 使 a[i]--, a[i+1]++, 请最小化极差
3 // 思路: 分别二分最小的最大值, 最大的最小值, 二者之差即为答案
4 #include <bits/stdc++.h>
5 using ll = long long;

```

```

6   using namespace std;
7   const int N = 200005;
8   int n;
9
10  bool check1(ll mid,vector<ll>&a){
11      ll cnt = 0;
12      for(int i = 1;i <= n;i++){
13          if(a[i] > mid) cnt+=a[i]-mid;
14          if(a[i] < mid) cnt = max((ll)0,cnt - (mid-a[i]));
15      }
16      return cnt == 0;
17  }
18
19  bool check2(ll mid,vector<ll>&a){
20      ll cnt = 0;
21      for(int i = 1;i <= n;i++){
22          if(a[i] > mid) cnt += a[i]-mid;
23          if(a[i] < mid) cnt -= mid-a[i];
24          if(cnt < 0) return 0;
25      }
26      return 1;
27  }
28
29  void sol(){
30      cin >> n;
31      vector<ll>a(n+1);
32      for_each(a.begin()+1,a.begin()+n+1,[](auto &x){cin >> x;});
33
34      ll l = 0,r = 1e18;
35      while(l < r){
36          ll mid = l+r>> 1;
37          if(check1(mid,a)) r=mid;
38          else l=mid+1;
39      }
40      ll ans1 = l;
41
42      l = 0,r = 1e18;
43      while(l < r){
44          ll mid = l+r+1 >> 1;
45          if(check2(mid,a)) l = mid;
46          else r = mid-1;
47      }
48      ll ans2 = l;
49      cout << ans1 - ans2 << endl;
50  }
51
52  int main() {
53      int T = 1; //fastio;
54      cin >> T;
55      while(T--){ sol(); }
56      return 0;
57  }

```

实数域二分

```
1 //一般实现为限制二分次数,或达到某个精度就停止
2 for(int ti = 1;ti <= 100;ti++){
3     double mid = (l + r) / 2;
4     if(check(mid)) r = mid;
5     else l = mid;
6 }
7 std::cout << l << '\n';
```

```
1 //例如: 求一个数的开三次方
2 const double eps = 1e-8;
3
4 double n;cin >> n;
5 double l = -10000,r = 10000;
6 while(r-l >= eps){
7     double mid = (l+r)/2;
8     if(mid*mid*mid >= n) r = mid;
9     else l = mid;
10 }
11 printf("%.6lf",r);
```

分数规划

[分数规划 - OI Wiki \(oi-wiki.org\)](https://oi-wiki.org/fractional-programming/)

分数规划用来求一个分式的极值。

每种物品有两个权值 a_i 和 b_i , 求一组 $w \in \{0,1\}$, 选出若干个物品使得 $\frac{\sum_{i=1}^n a_i * w_i}{\sum_{i=1}^n b_i * w_i}$ 最小/最大。

一般分数规划问题还会有一些奇怪的限制, 比如『分母至少为 k 』。

假如我们要求最大值, 二分check一个mid, 然后推柿子

$$\frac{\sum a_i * w_i}{\sum b_i * w_i} > mid$$

$$\Rightarrow \sum a_i * w_i - mid * \sum b_i * w_i > 0$$

$$\Rightarrow \sum w_i * (a_i - mid * b_i) > 0 \quad // \text{若不等式大于0说明mid可行}$$

主要难点在于如何求 $\sum w_i * (a_i - mid * b_i)$ 的最大值/最小值

```
1 //https://vjudge.net/problem/POJ-2976#author=0
2 //n门课至少选k门,使得所选sum(a)/sum(b)的平均学分最大
3 #include <iostream>
4 #include <vector>
5 #include <algorithm>
```



```

6   using namespace std;
7   const int N = 1003;
8   int n,k;
9   vector<double>a(N),b(N);
10
11  bool cmp(double x,double y){
12      return x > y;
13  }
14
15  bool check(double mid){//判断是否存在k个物品，其累计平均分>mid
16      double s = 0;
17      vector<double>c(n);
18
19      for(int i = 0;i < n;i++){
20          c[i] = a[i]-mid*b[i];//把c[i] = a[i]-mid*b[i]作为第i个物品的权值
21      }
22      sort(c.begin(),c.begin()+n,cmp);
23      for(int i = 0;i < k;i++){
24          s+=c[i];//贪心地选择权值最大的k个
25      }
26      return s > 0;
27  }
28
29  void uuz(){
30      for(int i = 0;i < n;i++){cin >> a[i];}
31      for(int i = 0;i < n;i++){cin >> b[i];}
32      double l = 0,r = 1e9;
33      while(r-l > 1e-6){
34          double mid = (l+r)/2;
35          //cout << mid << endl;
36          if(check(mid)) l = mid;
37          else r = mid;
38      }
39      int ans = r*100;
40      cout << ans+(r*100-ans >= 0.5) << endl;
41  }
42
43  int main(){
44      while(cin >> n >> k,n||k){
45          k = n-k;
46          uuz();
47      }
48  }

```

子段和问题

无长度限制

求一个子段，它的和最大，没有“长度大于等于F”这个限制

无长度限制的最大子段和问题是一个经典问题，只需要O(n)扫描该序列，不断地把新数加入子段，当子段和变为负数的时候，就把当前整个子段清空。扫描过程中出现过的最大的子段和即为所求

区间[l,r]的最大子段和：详见线段树

有长度限制

求一个子段，他的和最大，**长度不超过k**：[135. 最大子段和](#)

详见 [单调队列优化](#)

求一个子段，他的和最大，**长度不小于k**：

子段可以转化为前缀和相减的形式，则有

$$\max_{i-j \geq k} \{A_{j+1} + A_{j+2} + \dots + A_i\} = \max_{k \leq i \leq n} \{sum_i - \min_{0 \leq j \leq i-k} \{sum_j\}\}$$

```
1 //最佳牛围栏: https://www.acwing.com/problem/content/104/
2 #include <iostream>
3 #include <vector>
4 #include <algorithm>
5 using namespace std;
6 const int N = 100005;
7 int n, f;
8 int a[N];
9 double s[N];
10
11 bool check(double mid){//判断是否存在连续子段(长度>=f)，其最大平均数max_avg >= 当前mid
12     for(int i = 1; i <= n; i++){//相当于判断是否存在 max_avg - mid >= 0
13         s[i] = s[i-1] + (a[i] - mid); //a[i]-mid 的前缀和
14     }
15     double avg = 0;
16     for(int i = f; i <= n; i++){
17         avg = min(avg, s[i-f]);
```

```

18         if(s[i]-avg >= 0) return 1;//若s[i]-mins >= 0,说明存在子段[min_id,i]其平均
    数>=mid
19     }
20     return 0;
21 }
22
23 int main(){
24     cin >> n >> f;
25     for(int i = 1;i <= n;i++){cin >> a[i];}
26     double l = 0,r = 2000;
27     while(r-l > 1e-6){
28         double mid = (l+r)/2;
29         if(check(mid)) l = mid;//将问题转变为判定问题
30         else r = mid;
31     }
32     r*=1000;//选取r为结果
33     cout << (int)r << endl;
34 }

```

三分

如果需要求出**单峰函数**的极值点，通常使用二分法衍生出的三分法求单峰函数的极值点

三分法每次操作会舍去两侧区间中的其中一个。为减少三分法的操作次数，应使两侧区间尽可能大。因此，每一次操作时的 $lmid$ 和 $rmid$ 分别取 $mid - \varepsilon$ 和 $mid + \varepsilon$ 是一个不错的选择。

```

1 //https://www.luogu.com.cn/problem/P3382
2 //给出一个N次函数，保证在范围[l,r]内存在一点x，使得[l,x]上单调增，[x,r]上单调减。试求出x的值。
3 #include <iostream>
4 #include <cmath>
5 using namespace std;
6 const int N = 20;
7 const double eps = 1e-7;
8 int n;
9 double l,r,a[N];
10
11 double f(double x){
12     double ans = 0;
13     for(int i = n;i >= 0;i--){
14         ans+=a[i]*pow(x,i);
15     }
16     return ans;
17 }
18
19 int main(){

```

```

20     cin >> n >> l >> r;
21     for(int i = n; i >= 0; i--){ cin >> a[i]; }
22     while(r-l > eps){
23         double mid = (l+r)/2;
24         double lmid = mid - eps;
25         double rmid = mid + eps;
26         if(f(lmid) > f(rmid)) r = mid; // 诺要求凹函数峰值, 把符号换成 < 即可
27         else l = mid;
28     }
29     printf("%.61f", l);
30
31     return 0;
32 }

```

整数域三分

```

1  int l = 1, r = n;
2  while (l < r) { // 单峰函数求最小
3      int mid1 = l + (r - l) / 3;
4      int mid2 = r - (r - l) / 3;
5      if (f(mid1) < f(mid2)) r = mid2 - 1;
6      else l = mid1 + 1;
7  }
8  std::cout << f(l) << '\n';

```

双指针

```

1  for(int i = 1, j = 1; i <= n; i++){
2      while(j < i && check(i, j)) {
3          j++;
4          // 每道题的逻辑
5      }
6  }

```

一般可以将 $O(n^2)$ 或 $O(N \log N)$ 优化到 $O(n)$;

```

1  // 最长连续无重复子序列
2  #include <iostream>
3  using namespace std;
4  const int N = 1000006;
5  int a[N], s[N]; // s[a[i]] 判断 a[i] 出现的次数
6  int n, res;
7  int main() {
8      cin >> n;
9      for (int i = 0; i < n; i++)

```

```

10     cin >> a[i];
11
12     for (int i = 0, j = 0; i < n; i++){
13         s[a[i]] ++;
14         while (s[a[i]] > 1) {
15             //当右端点i的数出现次数>1, s[a[j]]--, 左端点j右移
16             s[a[j]]--;
17             j++;
18         }
19         res = max(res, i - j + 1);
20     }
21     cout << res << endl;
22     return 0;
23 }

```

[Median - POJ 3579 - Virtual Judge](#)

求 绝对值之差 $\leq$ mid 的数对个数

```

1 bool check(int mid){
2     long long ans = 0;
3     for(int i = 1, j = 1; i <= n; i++){//a[] 预先排好序
4         while(j <= n && a[j]-a[i] <= mid) j++;
5         j--;
6         ans += j-i;
7     }
8     return ans >= (m+1)/2;
9 }

```

排序

sort

需包含头文件

平均复杂度为 $O(N\log N)$, 不保证稳定性

语法: `sort(begin, end, cmp);`

其中begin为指向待sort()的数组的 第一个元素的指针, end为指向待sort()的数组的 最后一个元素的下一个位置的指针, cmp参数为排序准则(不写默认从小到大进行排序);

```

1 //默认从小到大:
2 sort(arr, arr+n); //数组下标从0开始
3 sort(arr+1, arr+n+1) //数组下标从1开始
4 sort(v.begin(), v.end());
5
6
7 //从大到小:
8 sort(str.begin(), str.end(), greater<int>());
9 sort(str.rbegin(), str.rend());
10 sort(str.begin(), str.end(), cmp); //bool cmp(int a, int b){return a > b;}以a,b大小
    为依据排序
11                                     //返回值为真, 则a排在b前面
12 sort(e+1, e+n+1, [](auto &x, auto &y){return x.a > y.b;}); //lambda表达式, 结构体数组以
    a为权重排序

```

stable_sort()

stable_sort() 也是一种 $O(n \log n)$ 时间复杂度的排序算法, 但它保证稳定性, 即相等的元素将保持其原始顺序。这使得它在需要保持相等元素顺序的场景中很有用。注意, 稳定排序的代价是额外的空间复杂度。

快速排序

不稳定

快速排序的最优时间复杂度和平均时间复杂度为 $O(n \log n)$, 最坏时间复杂度为 $O(n^2)$ 。

```

1 #include<iostream>
2 using namespace std;
3 const int N = 1e6 + 5;
4 int n;
5 int q[N];
6 void quick_sort(int q[], int l, int r) {
7     if (l >= r) return;
8     int x = q[l], i = l - 1, j = r + 1;
9     while (i < j) {
10         do i++; while (q[i] < x);
11         do j--; while (q[j] > x);
12         if (i < j) swap(q[i], q[j]);
13     }
14     quick_sort(q, l, j); //注意边界问题, 必须以j为基准
15     quick_sort(q, j + 1, r);
16 }
17 int main() {
18     scanf("%d", &n);

```

```

19     for (int i = 0; i < n; i++) scanf("%d", &q[i]);
20
21     quick_sort(q, 0, n - 1);
22
23     for (int i = 0; i < n; i++) printf("%d", q[i]);
24 }

```

第k小的数

```

1 //https://www.luogu.com.cn/problem/P1923
2 //k从第0小开始，诺要从1开始则++k即可，时间复杂度O(N)
3 #include<iostream>
4 using namespace std;
5 const int N = 5e6 + 5;
6 int n,k;
7 int q[N];
8 void quick_sort(int q[], int l, int r) {
9     if (l >= r) return;
10    int x = q[l], i = l - 1, j = r + 1;
11    while (i < j) {
12        do i++; while (q[i] < x);
13        do j--; while (q[j] > x);
14        if (i < j) swap(q[i], q[j]);
15    }
16    if(k <= j) quick_sort(q, l, j); //每次判断条件,只需排一边
17    else quick_sort(q, j + 1, r);
18 }
19 int main() {
20     scanf("%d %d", &n,&k);
21     for (int i = 0; i < n; i++) scanf("%d", &q[i]);
22     quick_sort(q, 0, n - 1);
23     //stl函数实现了类似功能:nth_element(q,q+k,q+n);
24     //诺下标从1开始:++k; nth_element(a+1,a+k,a+n+1);
25     cout << q[k];
26 }

```

归并排序

稳定

时间复杂度在最优、最坏与平均情况下均为 $\Theta(n \log n)$ ，空间复杂度为 $\Theta(n)$ 。

先排左半边，再排右半边，最后合并。

```

1 void merge_sort(int q[], int l, int r){
2     //递归的终止条件
3     if(l >= r) return;
4

```

```

5 //第一步：分成子问题
6 int mid = l + r >> 1;
7
8 //第二步：递归处理子问题
9 merge_sort(q, l, mid), merge_sort(q, mid + 1, r);
10
11 //第三步：合并子问题
12 int i = l, j = mid + 1, k = 0, tmp[r - l + 1];
13 while(i <= mid && j <= r){
14     if(q[i] <= q[j]) tmp[k++] = q[i++];
15     else tmp[k++] = q[j++];
16 }
17 while(i <= mid) tmp[k++] = q[i++];
18 while(j <= r) tmp[k++] = q[j++];
19
20 for(k = 0, i = l; i <= r; k++, i++) q[i] = tmp[k];
21 }

```

逆序对

```

1 //https://www.luogu.com.cn/problem/P1908
2 //O(NlogN)归并排序求逆序对数量、冒泡排序需要交换的次数
3 //也可离散化后用树状数组/线段树求逆序对
4 #include <iostream>
5 using namespace std;
6 const int N = 500005;
7 int n,a[N];
8 long long ans;
9
10 void merge_sort(int a[],int l,int r){
11     if(l == r) return;
12     int mid = l + r >> 1;
13     merge_sort(a,l,mid);merge_sort(a,mid+1,r);
14     int i = l,j = mid+1,t[r-l+1],k = 0;
15     while(i <= mid && j <= r){
16         if(a[i] <= a[j]) t[k++] = a[i++];
17         else t[k++] = a[j++],ans += mid-i+1;//核心代码
18     }
19     while(i <= mid) t[k++] = a[i++];
20     while(j <= r) t[k++] = a[j++];
21     for(k = 0,i = l;i <= r;k++,i++){
22         a[i] = t[k];
23     }
24 }
25
26 int main(){
27     cin >> n;
28     for(int i = 1;i <= n;i++){ scanf("%d",&a[i]); }
29     merge_sort(a,1,n);
30     cout << ans;

```



```

1 //奇数码问题 https://www.acwing.com/problem/content/110/
2 //n*n (n为奇数) 二维按行化为一维(不包括0), 归并排序1 ~ n*n-1求逆序对数量;
3 //诺两个数码逆序对奇偶性相同则可以转变。

```

对于一个排列，交换任意两个元素的位置，必然改变逆序对奇偶性。

分治

就是把一个复杂的问题分成两个或更多的相同或相似的子问题，直到最后子问题可以简单的直接求解，原问题的解即子问题的解的合并。

大概的流程可以分为三步：分解 -> 解决 -> 合并。

1. 分解原问题为结构相同的子问题。
2. 分解到某个容易求解的边界之后，进行递归求解。
3. 将子问题的解合并成原问题的解。

分治法能解决的问题一般有如下特征：

- 该问题的规模缩小到一定的程度就可以容易地解决。
- 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质，利用该问题分解出的子问题的解可以合并为该问题的解。
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

[P7883 平面最近点对（加强加强版） - 洛谷 \(luogu.com.cn\)](#)

给定平面内 n 个点 $p(x,y)$ ，求距离最近的两个点的距离(的平方)。由于输入的点为整点，因此这个值一定是整数。

将所有点排序后，将整个区间一分为二，分别递归处理， $d = \min(sol(S1), sol(S2))$ ，再处理两个区间交界范围的点，假设当前交界点 x 坐标为 $midx$ ，我们只需要处理 $x \in [midx - d, midx + d]$ 范围内的点即可。将这个区域的点按照 y 坐标升序排列，然后二重循环枚举 (p_i, p_j) ， $d = \min(d, dist(p_i, p_j))$ ，如果当前两个点之间的 $\Delta y \geq d$ ，则可以停止枚举第二个点。

```

1  #include <iostream>
2  #include <algorithm>
3  using namespace std;
4  const int N = 400005;
5  int b[N];
6
7  struct node{
8      long long x,y;
9      bool operator < (const node& e2){
10         return x != e2.x ? x < e2.x : y < e2.y;

```

```

11     }
12 }a[N];
13
14 long long pow2(long long x) {return x*x;}
15
16 long long dist(node p1,node p2){
17     return pow2(p1.x-p2.x) + pow2(p1.y-p2.y);
18 }
19
20 void merge(int l,int r){
21     int mid = l + r >> 1;
22     int i = l,j = mid+1,k = 0;
23     node temp[r-l+1];
24
25     while(i <= mid && j <= r){
26         if(a[i].y < a[j].y) temp[k++] = a[i++];
27         else temp[k++] = a[j++];
28     }
29     while(i <= mid) temp[k++] = a[i++];
30     while(j <= r) temp[k++] = a[j++];
31
32     for(i = l,k = 0;i <= r;i++,k++){
33         a[i] = temp[k];
34     }
35 }
36
37 long long sol(int l,int r){
38     if(l >= r) return 8e18;
39     if(l+1 == r){//边界: 当前区间仅有两个点, 直接计算即可。
40         merge(l,r);
41         return dist(a[l],a[r]);
42     }
43     int mid = l + r >> 1;
44     long long midx = a[mid].x,cnt = 0;//因为合并时a[mid]会变,需要提前记录
45     long long d = min(sol(l,mid),sol(mid+1,r));
46     merge(l,r);
47     for(int i = l;i <= r;i++){
48         if(pow2(a[i].x - midx) < d){
49             b[++cnt] = i;
50         }
51     }
52     for(int i = 1;i <= cnt;i++){
53         for(int j = i+1;j <= cnt && pow2(a[b[i]].y - a[b[j]].y) < d;j++){
54             d = min(d,dist(a[b[i]],a[b[j]]));
55         }
56     }
57     return d;
58 }
59
60 int main(){
61     int n;cin >> n;
62     for(int i = 1;i <= n;i++){

```

```

63     cin >> a[i].x >> a[i].y;
64 }
65 sort(a+1,a+n+1);
66 cout << sol(1,n);
67 // printf("%.4lf",sqrt(sol(1,n)));
68 }

```

二进制

```

1 //求n的二进制第k位 从0开始数
2 #include <iostream>
3 using namespace std;
4 //10 = (1010)2
5 int main() {
6     int n, k;
7     cin >> n >> k;
8     cout << (n >> k & 1); //右k移位, 再&1, k的值不会改变
9     return 0;
10 }

```

lowbit

求n的二进制 第一次出现的1对应的值:

368:

n = 1011==1==0000

-n = 010001111

-n+1 = 0100==1==0000 //以补码形式储存

n&-n = 10000 = 16

```

1 int lowbit(int x) {return x&-x;}

```

```

1 //upbit
2 long long upbit(long long x) { return x ? 1LL << (63 - __builtin_clz11(x)) : 0; }
3 long long upbit(long long x) { return x ? 1LL << (long long)log2(x) : 0; }

```

lowbit的应用:

```

1 //求n的二进制中出现了多少个1
2 int lowbit(int n) { return n & -n; }
3
4 int main() {
5     int n,res = 0; cin >> n;
6     while (n) {
7         n -= lowbit(n);
8         res++;
9     }
10    cout << res << endl;
11 }
12 //gnu编译器实现,性能最快,接近硬件极限
13 int res = __builtin_popcount(x);

```

```

1 //判断一个数是不是2的倍数
2 int lowbit(int n) {return n&-n;}
3
4 int main(){
5     if(n == lowbit(n)) "yes"
6     else "no"
7 }

```

求正整数n的二进制长度

$(100101)_2 = 6$

```

1 int len = log2(x)+1;

```

求正整数n的二进制第k位代表的十进制数 (2^k 或0)

```

1 long long deg(long long num, int deg) { return num & (1LL << deg); }
2 //25 = {1,0,0,8,16}

```

状态压缩

用二进制表示状态

例如, $a[] = \{1,2,3,4,5,6,7,8\}$;

则v[22]表示10110 = $a[5]+a[3]+a[2]$

二进制枚举子集, 时间复杂度 $O(n \times 2^n)$, 空间复杂度 $O(1)$

```

1 //二进制枚举子集
2 for (int i = 0; i < (1 << n); i++) {
3     for (int j = 0; j < n; j++) {
4         if (i >> j & 1) {
5             now += a[j];
6         }
7     }
8     ans = max(ans, now);
9 }

```

dfs枚举子集，时间复杂度更低 $O(2^n)$ ，需要额外 $O(n)$ 的栈空间，剪枝能力更强

```

1 //dfs枚举子集
2 void dfs(int idx, int sum) {
3     if (idx == n) {
4         ans = max(ans, sum);
5         return;
6     }
7     dfs(idx + 1, sum);           // 不选
8     dfs(idx + 1, sum + a[idx]); // 选
9 }

```

95. 费解的开关 - AcWing题库

5 * 5 的方格，每个方格都有一盏灯和一个开关，按下开关后当前方格和与其相邻的方格的灯状态会发生变化。

灯的状态用1代表亮着，0代表关着。给定所有灯初始状态，判断能否在6步内使所有灯都变亮。

先固定第一行状态（暴力枚举00000~11111），每一行的暗灯都由下面一行去点亮，此时已经固定了当前状态的最终答案

再枚举第 $i=(2,3,4,5)$ 行，若第 $a[i-1][j]$ 为0，则需要 $turn[i][j]$ 开关，统计能否点亮所有灯和turn的次数即可

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int N = 7;
4 bool a[N][N], b[N][N];
5 int di[] = {0, -1, 1, 0, 0}, dj[] = {0, 0, 0, -1, 1};
6 int cnt, ans;
7
8 void turn(int i, int j) {
9     for (int w = 0; w < 5; w++) {

```

```

10     int x = i + di[w], y = j + dj[w];
11     if(x >= 1 && x <= 5 && y >= 1 && y <= 5){
12         if(a[x][y]){cnt--;}
13         else {cnt++;}
14         a[x][y]^=1;
15     }
16 }
17 }
18
19 void sol(){
20     ans = 0x3f3f3f3f, cnt = 0;
21     for(int i = 1; i <= 5; i++){
22         string s; cin >> s;
23         for(int j = 1; j <= 5; j++){
24             a[i][j] = s[j-1] - '0';
25             if(a[i][j]) cnt++;
26         }
27     }
28     memcpy(b, a, sizeof b);
29     int temp = cnt;
30
31     for(int q = 0; q < 1 << 5; q++){//q表示第一行状态,从00000~11111
32         cnt = temp;//cnt统计亮灯的个数
33         int step = 0;//step记录开关操作次数
34
35         for(int k = 0; k < 5; k++){
36             if(q >> k & 1){ step++; turn(1, k+1); }
37         }//用当前状态q固定第一行
38
39         for(int i = 2; i <= 5; i++){//枚举第2~5行
40             if(step > 6) break;
41             for(int j = 1; j <= 5; j++){
42                 if(a[i-1][j] == 0){ step++; turn(i, j); }//每一行的暗灯都由下面一行去
点亮
43                 if(cnt == 25){ ans = min(ans, step); }//更新答案
44             }
45         }
46         memcpy(a, b, sizeof a);//还原数组a
47     }
48
49     if(ans > 6) cout << -1 << endl;
50     else cout << ans << endl;
51 }
52
53 int main(){
54     int tt; cin >> tt;
55     while(tt--){sol();}
56     return 0;
57 }

```

枚举子掩码

枚举单个掩码m的所有子掩码时间复杂度 $O(2^k)$ ，其中k为掩码m中1的位数

```
1 int m = 0b1101; // 降序枚举状态m的子掩码
2 for (int s = m; s; s = (s - 1) & m) { // s也可以从m的任意一个子掩码开始
3     //cout << bitset<4>(s) << '\n';
4 }
```

枚举所有掩码的子掩码的时间复杂度 $O(3^n)$ ，其中n为全集位数。常用于子集DP

```
1 int cnt = 0;
2 int n = 15;
3 for (int m = 0; m < 1 << n; m++) {
4     for (int s = m; s; s = (s - 1) & m) {
5         //cout << bitset<15>(s) << '\n';
6     }
7 }
```

枚举超集

时间复杂度 $O(2^{n-k})$ ，其中n为全集位数，k为固定掩码中1的个数

```
1 int max_state = 0b11111;
2 int t = 0b101; // 升序枚举状态t的超集
3 for (int s = t; s < max_state; s = (s + 1) | t) {
4     cout << bitset<5>(s) << '\n';
5 }
```

位运算

$a+b = a|b + a\&b = a\^b + (a\&b)*2$

$a-b = (a\&\sim b) - (b\&\sim a)$

^ 异或

相同为0，不同为1，可以看做不进位加法

$$0 \wedge 0 = 0$$

$$1 \wedge 0 = 1$$

$$0 \wedge 1 = 1$$

$$1 \wedge 1 = 0$$

$$A \wedge A = 0$$

$$A \wedge 0 = A$$

$$a \wedge b \wedge b = a$$

自反性

$$a \wedge b \wedge c \wedge d = b \wedge d \wedge a \wedge c$$

无序性

$a \wedge b = c$ 可移项为 $a = b \wedge c$ ，移项时无需改变符号

如果i为偶数，则有 $i \wedge (i+1) = 1$

比较两数值是否同号， $a \wedge b > 0$ 替换掉 $a * b > 0$

$a \wedge= b \wedge= a \wedge= b$ 相当于 `swap(a,b)`

判断成对的数中只出现一次的数

```
1 //给定一个非空整数数组，除了某个元素只出现一次以外，其余每个元素均出现**两次**。找出那个只出现了一次的元素。
2 int ans = 0;
3 for(int i = 1; i <= n; i++) {ans ^= a[i];}
4 cout << ans;
```

异或前缀和

```
1 for(int i = 1; i <= n; i++){
2     s[i] = s[i-1]^a[i];
3 }
4
5 int query(int l, int r){
6     return s[r]^s[l-1];
7 }
```

如果 $(n-1) \% 4 == 0$ ，则 $0 \sim n-1$ 的异或前缀和为0， $s[0, 3, 7, 11, 15 \dots] = 0$

& 与

判断奇偶

```
1 | if (n & 1) { } //则为奇数
```

移除低位中第一个1

```
1 | x & (x-1); //相当于x - lowbit(x)
```

>> 移位

$n \gg k$ 相当于 $n/2^k$, 向下取整

$n \ll k$ 相当于 $n \times 2^k$

```
1 | //十进制转二进制
2 | int n; cin >> n; //32位
3 | for (int i = 31; i >= 0; i--) {
4 |     cout << (n >> i & (1));
5 | }
```

离散化

数组下标跨度长, 但只用到了部分下标

```
1 | vector<int>arr; //储存所有待离散化的值
2 | sort(arr.begin(),arr.end()); //将所有值排序
3 | arr.erase(unique(arr.begin(),arr.end()),arr.end()); //去掉重复元素
4 |
5 | int find(int x){
6 |     return lower_bound(arr.begin(),arr.end(),x)-arr.begin()+1; //加1从下标为1开始映射, 不加则从0开始
7 | }
```

```

1  for(int i = 1;i <= n;i++){
2      cin >> a[i];
3      hs[i] = a[i];
4  }
5  sort(hs+1,hs+n+1);
6  int m = unique(hs+1,hs+n+1) - hs - 1;//a[1~m]即为去重后的元素(离散至1~m(<=n)),不去重也
   没关系(离散至1~n)
7  for(int i = 1;i <= n;i++){
8      a[i] = lower_bound(hs+1,hs+m+1,a[i]) - hs + 1;//a[i]即为离散后的数值,hs[a[i]]为
   a[i]原来的值
9  }

```

部分涉及区间染色的题目，对于区间 $[l, r]$ 离散化时，需要将 $[l-1, r-1]$ 也一起加入离散化数组并去重，否则可能出现以下情况：例如用区间 $[1, 3]$ 和 $[6, 10]$ 覆盖区间 $[1, 10]$ 时，常规离散化会导致 $[1, 10]$ 整个区间被覆盖。例题：[P3740 [HAOI2014](https://www.luogu.com.cn/problem/HAOI2014)] [贴海报 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/HAOI2014)

搜索

DFS

深度优先搜索，递归实现

在深度优先搜索中，对于最新发现的顶点，如果它还有以此为顶点而未探测到的边，就沿此边继续探测下去，当顶点 v 的所有边都已被探寻过后，搜索将回溯到发现顶点 v 有起始点的那些边。这一过程一直进行到已发现从源顶点可达的所有顶点为止。如果还存在未被发现的顶点，则选择其中一个作为源顶点，并重复上述过程。整个过程反复进行，直到所有的顶点都被发现时为止。

括号序列

```

1  //按字典序输出长度为n的所有合法括号序列 1 <= n <= 20
2  //https://vjudge.net/problem/AtCoder-typical90_b#author=DeepSeek_zh
3  #include <iostream>
4  using namespace std;
5  int n;
6
7  void dfs(string s,int l,int r){//当前序列为s,且用了l个左括号的r个右括号
8      if(l == n && r == n){
9          cout << s << '\n';
10         return ;
11     }
12     if(l < n) dfs(s+'(',l+1,r);
13     if(r < l) dfs(s+')',l,r+1);//对于所有位置i而言,其左括号的数量>=右括号的数量
14 }
15
16 int main(){
17     cin >> n;

```

```

18     if(n&1) return 0;
19     n/=2;
20     dfs("",0,0);
21 }

```

全排列

```

1 //n选m的全排列,标记数组写法
2 void dfs(int cnt){//dfs(1)
3     if(cnt > m){
4         for(int i = 1;i <= m;i++){
5             cout << path[i] << ' ';
6         }
7         cout << '\n';
8         return;
9     }
10    for(int i = 1;i <= n;i++){
11        if(!st[i]){
12            path[cnt] = i;
13            st[i] = 1;
14            dfs(cnt+1);
15            st[i] = 0;
16        }
17    }
18 }

```

```

1 //n选m的全排列,state表示二进制状态替换标记数组 l_idx
2 void dfs(int state,int cnt){//dfs(1,1)
3     if(cnt > m){
4         for(int i = 1;i <= m;i++){
5             cout << path[i] << ' ';
6         }
7         cout << '\n';
8         return;
9     }
10    for(int i = 1;i <= n;i++){
11        if(state>>i&1) continue;
12        path[cnt] = i;
13        dfs(state|1<<i,cnt+1);
14    }
15 }
16
17 //n选m的全组合 l_idx
18 void dfs(int u,int cnt){//dfs(1,1)
19     if(cnt > m){
20         for(int i = 1;i <= m;i++){
21             cout << path[i] << ' ';
22         }
23         cout << '\n';
24         return;
25     }

```

```

26     for(int i = u; i <= n; i++){
27         path[cnt] = i;
28         dfs(i+1, cnt+1);
29     }
30 }

```

```

1 //n选m的全排列&全组合(1~n)
2 //(t.path[]可以从任意合法排列开始)
3 #include <iostream>
4 #include <vector>
5 using namespace std;
6
7 struct PC{//PC t(n,m);   l_idx
8     int n,m;
9     vector<int>path;
10    PC(int n,int m){
11        this->n = n;this->m = m;
12        path.resize(n+1);
13        for(int i = 1;i <= n;i++) path[i] = i;
14    }
15
16    bool next_comb(){
17        int j = m;
18        while (j >= 1 && path[j] == n - m + j) j--;
19        if (j == 0) return 0;
20        path[j]++;
21        for (int i = j + 1; i <= m; i++) path[i] = path[i - 1] + 1;
22        return 1;
23    }
24
25    bool next_perm(){
26        for (int i = m,used = 0; i >= 1; i--,used = 0) {
27            for (int j = 1; j < i; ++j) used |= 1 << path[j];
28            for (int x = path[i]+1;x <= n;x++){
29                if((used >> x & 1)) continue;
30                path[i] = x; used = ~ (used | 1 << x) ^ 1;
31                for(int j = i+1;j <= m;j++){
32                    int w = __builtin_ctz(used);
33                    path[j] = w;
34                    used ^= 1 << w;
35                }
36                return 1;
37            }
38        }
39        return 0;
40    }
41 };
42
43 int main(){

```

```

44     int n,m; cin >> n >> m;
45     PC t(n,m); //初始化
46     do{
47         for(int i = 1;i <= t.m;i++){
48             cout << t.path[i] << ' ';
49         }
50         cout << '\n';
51     }while(t.next_perm());
52 }

```

剪枝

优化搜索顺序

搜索前排序,减小搜索规模、优先搜索分支较少的节点

排除等效冗余

如果沿着几条不同分支到达的子树是等效的,那么只需要对其中一条分支进行搜索

可行性剪枝 (上下界剪枝)

对当前状态进行检查,如果当前分支已经无法到达递归的边界,就进行回溯

最优化剪枝

在最优化问题的搜索过程中,如果当前花费的代价超过了已经搜到的最优解,此时可以停止搜索,直接回溯

记忆化

记录每个状态的搜索结果,在重复遍历一个状态时直接检索并返回

其它优化

使用数据结构、状态压缩、位运算等优化

```

1  //导弹防御系统 https://www.luogu.com.cn/problem/P10490
2  //dfs+剪枝+全局最小值
3  #include <iostream>
4  using namespace std;
5  const int N = 55;
6  int n;
7  int arr[N];
8  int up[N],down[N];
9  int ans;
10
11 void dfs(int u,int su,int sd){ //前u个导弹,已经使用su个up系统,sd个down系统
12     if(su + sd >= ans) return; //如果已经超过了当前最优答案,直接剪枝

```

```

13     if(u == n){ans = su + sd;return;}//所有导弹都考虑过了,并且没有超过当前最优答案,则更新最优答案
14
15     //方案一: 使用up系统拦截第u个导弹
16     int i = 0;
17     while(i < su && up[i] >= arr[u]) i++;//贪心找到当前低于u导弹的最高的up系统i (up[]为降序数组)
18     int temp = up[i];//temp用于回溯时恢复现场
19     up[i] = arr[u];
20     if(i >= su)dfs(u+1,su+1,sd);//若i没找到则新建一个up系统拦截
21     else dfs(u+1,su,sd);//否则使用原来的up系统拦截
22     up[i] = temp;//恢复现场
23
24     //方案二: 使用down系统拦截第u个导弹
25     i = 0;
26     while(i < sd && down[i] <= arr[u]) i++;
27     temp = down[i];
28     down[i] = arr[u];
29     if(i >= sd)dfs(u+1,su,sd+1);
30     else dfs(u+1,su,sd);
31     down[i] = temp;
32 }
33
34 int main(){
35     while(cin >> n,n){
36         ans = n;
37         for(int i = 0;i < n;i++){ cin >> arr[i]; }
38         dfs(0,0,0);
39         cout << ans <<endl;
40     }
41 }

```

```

1 //数独 https://www.luogu.com.cn/problem/P10481
2 #include <iostream>
3 using namespace std;
4 const int N = 9,M = 1 << N;
5 int a[N][N];
6 int mp[M];//预处理每个lowbit对应的数
7 int ones[M];//预处理每个状态可填的数,优先搜索分支最小的状态
8 int sx[N],sy[N],sz[N];//状态压缩,表示每行、每列、每块方格还能填的数
9
10 int get(int x,int y){
11     return sx[x]&sy[y]&sz[x/3*3+y/3];
12 }
13
14 bool dfs(int cnt){
15     if(!cnt) return 1;
16     int mn = 10;
17     int x,y;

```

```

18     for(int i = 0; i < N; i++){//优先找分支较少的进行拓展
19         for(int j = 0; j < N; j++){
20             if(a[i][j] == 0){
21                 int now = ones[get(i,j)];
22                 if(now < mn){
23                     mn = now;
24                     x = i, y = j;
25                 }
26             }
27         }
28     }
29     for(int i = get(x,y); i; i -= i&-i){
30         int k = mp[i&-i];
31         a[x][y] = k+1;
32         sx[x] ^= 1 << k;
33         sy[y] ^= 1 << k;
34         sz[x/3*3+y/3] ^= 1 << k;
35
36         if(dfs(cnt-1)) return 1;
37
38         a[x][y] = 0;
39         sx[x] ^= 1 << k;
40         sy[y] ^= 1 << k;
41         sz[x/3*3+y/3] ^= 1 << k;
42     }
43     return 0;
44 }
45
46 void sol(){
47     for(int i = 0; i < N; i++) sx[i] = sy[i] = sz[i] = (1<<N)-1;
48     int cnt = 0;
49     for(int i = 0; i < N; i++){
50         for(int j = 0; j < N; j++){
51             if(a[i][j]){
52                 int k = a[i][j]-1;
53                 sx[i] ^= 1 << k;
54                 sy[j] ^= 1 << k;
55                 sz[i/3*3+j/3] ^= 1 << k;
56             }
57             else cnt++;
58         }
59     }
60     dfs(cnt);
61 }
62
63 int main(){
64     for(int i = 0; i < N; i++) mp[1<<i] = i;
65     for(int i = 0; i < 1 << N; i++){
66         for(int j = i; j; j -= j&-j){
67             ones[i]++;
68         }
69     }

```

```

70     int mt;cin >> mt;
71     while(mt-->0) {
72         for(int i = 0;i < N;i++){
73             string s;cin >> s;
74             for(int j = 0;j < N;j++){
75                 char x = s[j];
76                 if(x == '.') a[i][j] = 0;
77                 else a[i][j] = x - '0';
78             }
79         }
80         sol();
81         for(int i = 0;i < N;i++){
82             for(int j = 0;j < N;j++){
83                 cout << a[i][j];
84             }cout << '\n';
85         }
86     }
87 }

```

迭代加深

即**每次限制搜索深度**的深度优先搜索。

迭代加深搜索的本质还是深度优先搜索，只不过在搜索的同时带上了一个深度，当达到设定的深度时就返回，一般用于找最优解。如果一次搜索没有找到合法的解，就让设定的深度加一，重新从根开始。

当状态随着深度增加比较多时，BFS队列的空间复杂度会很大。当BFS在空间上不够优秀，而且问题要找最优解时，应该考虑迭代加深搜索，迭代加深也可以近似看做BFS。

过程

首先设定一个较小的深度作为全局变量，进行DFS。每进入一次DFS，将当前深度加一，当发现 d 大于设定的深度 $limit$ 就返回。如果在搜索的途中发现了答案就可以回溯，同时在回溯的过程中可以记录路径。如果没有发现答案，就返回到函数入口，增加设定深度，继续搜索。

```

1  IDDFS(u,d)
2      if (d > limit) return
3      for each edge (u,v){
4          IDDFS(v,d+1)
5      }
6      return

```


BFS

广度优先搜索，**队列**实现，求最短路径

每次都尝试访问同一层的节点。如果同一层都访问完了，再访问下一层。

```
1 //走迷宫 https://www.acwing.com/problem/content/description/846/
2 #include <iostream>
3 #include <cstring>
4 #include <queue>
5 using namespace std;
6 const int N = 110;
7 int n,m;
8 int g[N][N]; //存图
9 int d[N][N]; //存点到起点的距离
10 queue<pair<int, int>>q; //队列实现
11
12 int dfs() {
13     memset(d, -1, sizeof d); //初始化为-1表示没有走过
14     q.push({ 0,0 }); //表示起点走过了
15     d[0][0] = 0;
16
17     int dx[4] = { -1,1,0,0 }, dy[4] = { 0,0,-1,1 };
18     while (q.size()) { //队列不为空
19         auto t = q.front(); //取队头
20         q.pop();
21         for (int i = 0; i < 4; i++) { //枚举四个方向,扩展t
22             int x = t.first + dx[i], y = t.second + dy[i];
23             if (x >= 0 && x < n && y >= 0 && y < m && d[x][y] == -1 && g[x][y]
== 0) {
24                 //在边界内 是空地 且之前没有走过
25                 d[x][y] = d[t.first][t.second] + 1; //则到起点的距离+1
26                 q.push({ x,y }); //新坐标入队
27             }
28         }
29     }
30     return d[n - 1][m - 1];
31 }
32
33 int main() {
34     cin >> n >> m;
35     for (int i = 0; i < n; i++) {
36         for (int j = 0; j < m; j++) {
37             cin >> g[i][j];
38         }
39     }
40     cout << dfs();
41 }
```

双端队列BFS

时间复杂度为严格的 $O(N+M)$

对于一张边权为0或1的无向图。搜索时如果边权为0则将该新节点从队头入队，如果为1则从队尾入队

```
1 //电车 https://www.acwing.com/problem/content/4252/
2 #include <iostream>
3 #include <cstring>
4 #include <deque>
5
6 const int N = 105,M = N*N;
7 int n,s,t;
8 int h[N],e[M],ne[M],w[M],idx;
9 int dist[N];
10 bool vis[N];
11
12 void add(int a,int b,int c){
13     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
14 }
15
16 void uuz(){
17     std::memset(dist,0x3f,sizeof dist);
18     std::deque<int>dq;
19     dq.push_back(s);
20     dist[s] = 0;
21     while(dq.size()){
22         int x = dq.front();
23         dq.pop_front();
24         if(x == t) return;
25         for(int i = h[x];~i;i = ne[i]){
26             int y = e[i];
27             if(dist[y] > dist[x] + w[i]){
28                 dist[y] = dist[x] + w[i];
29                 if(w[i] == 0) { dq.push_front(y); }
30                 else { dq.push_back(y); }
31             }
32         }
33     }
34 }
35
36 int main(){
37     std::memset(h,-1,sizeof h);
38     std::cin >> n >> s >> t;
39     for(int i = 1;i <= n;i++){
40         int k;
41         std::cin >> k;
42         for(int j = 1;j <= k;j++){
43             int x;std::cin >> x;
44             if(j == 1) add(i,x,0);
```

```

45         else add(i,x,1);
46     }
47 }
48 uuz();
49 if(dist[t] == 0x3f3f3f3f) std::cout << -1;
50 else std::cout << dist[t];
51 }

```

[P1948 [USACO08JAN](#)] Telephone Lines S - 洛谷 ([luogu.com.cn](#))

求出一条路径，使点1到n中第k+1长的边最短，输出这条边的长度。
 二分一个答案x，搜索时若当前w > x则权值看作1，代表由通信公司免费报销。
 若最终需要报销的边数 <= k 则说明当前 x 符合条件。

```

1  #include <iostream>
2  #include <cstring>
3  #include <deque>
4  using namespace std;
5  const int N = 1003,M = 20004;
6  int n,m,k;
7  int h[N],e[M],ne[M],w[M],idx;
8  int dist[N];
9
10 void add(int a,int b,int c){
11     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
12 }
13
14 bool check(int x){
15     memset(dist,0x3f,sizeof dist);
16     dist[1] = 0;
17     deque<int>q;
18     q.push_back(1);
19     while(q.size()){
20         int t = q.front();
21         q.pop_front();
22         for(int i = h[t];~i;i = ne[i]){
23             int k = e[i];
24             int val = w[i] > x ? 1 : 0;
25             if(dist[k] > dist[t] + val){
26                 dist[k] = dist[t] + val;
27                 if(val) q.push_back(k);
28                 else q.push_front(k);
29             }
30         }
31     }
32     return dist[n] <= k;
33 }
34
35 int main(){
36     memset(h,-1,sizeof h);

```

```

37     cin >> n >> m >> k;
38
39     for(int i = 1; i <= m; i++){
40         int a,b,c; cin >> a >> b >> c;
41         add(a,b,c); add(b,a,c);
42     }
43
44     int l = 0, r = 1e6;
45     while(l < r){
46         int mid = l + r >> 1;
47         if(check(mid)) r = mid;
48         else l = mid + 1;
49     }
50     if(dist[n] == 0x3f3f3f3f) cout << -1;
51     else cout << l;
52 }

```

[Maze Challenge with Lack of Sleep \(★4\) - AtCoder typical90_aq - Virtual Judge \(vjudge.net\)](#)

给定一个 $N \times M$ 的网格，`#`代表障碍，`.`代表路径，求从起点 (rs, cs) 到终点 (rt, ct) ，最少转向次数。

设定状态 (x, y, dir) 表示当前位置为 (x, y) ，方向为 dir ，拓展到下一个节点 (nx, ny, i) 时，如果 dir 与 i 方向一致，则入队首，否则入队尾。

```

1  #include <iostream>
2  #include <cstring>
3  #include <queue>
4  using namespace std;
5  const int N = 1003;
6  int n, m, rs, cs, rt, ct;
7  string s[N];
8  int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};
9  int dist[N][N][4];
10 int ans = 0x3f3f3f3f;
11
12 struct node{
13     int x, y, dir;
14 };
15
16 void uuz(){
17     memset(dist, 0x3f, sizeof dist);
18     deque<node> q;
19     for(int i = 0; i < 4; i++){
20         q.push_back(node{rs, cs, i});
21         dist[rs][cs][i] = 0;
22     }
23     while(q.size()){
24         auto [x, y, dir] = q.front();
25         q.pop_front();

```

```

26         if(x == rt && y == ct) ans = min(ans,dist[x][y][dir]);
27
28
29         for(int i = 0;i < 4;i++){
30             int nx = x + dx[i],ny = y + dy[i];
31             if(nx < 1 || nx > n || ny < 1 || ny > m || s[nx][ny] == '#')
continue;
32             if(dist[nx][ny][i] > dist[x][y][dir] + (i != dir)){
33                 dist[nx][ny][i] = dist[x][y][dir] + (i != dir);
34                 if(i == dir)q.push_front(node{nx,ny,i});
35                 else q.push_back(node{nx,ny,i});
36             }
37         }
38     }
39 }
40
41 int main(){
42     cin >> n >> m;
43     cin >> rs >> cs >> rt >> ct;
44     for(int i = 1;i <= n;i++){
45         cin >> s[i];s[i] = ' ' + s[i];
46     }
47     uuz();
48     cout << ans;
49 }

```

优先队列BFS

每次从队列中取出当前代价状态最小的进行扩展，当每个状态第一次从队列中被取出时，就得到了起点到该状态的最小状态，之后再被取出时则可直接忽略。

[UVA11367 Full Tank? - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/UVA11367)

有N个城市M条无向边，每个城市都有一个加油站，加每单位油花费p[i]。
回答q次询问，每次给定油箱容量为c，起始点为st，终点为ed。求st到ed的最小花费？

```

1  #include <iostream>
2  #include <vector>
3  #include <cstring>
4  #include <queue>
5  using namespace std;
6  const int N = 1003,M = 20004;
7  int n,m,q;
8  int p[N];
9  int h[N],e[M],ne[M],w[M],idx;
10
11 struct node{
12     int x,oil,cost;

```

```

13     bool operator < (const node &e2) const {
14         return cost > e2.cost; //小根堆, cost小的在堆顶, 每次用花费最小的进行拓展, 类似
dijkstra
15     }
16 };
17
18 void add(int a, int b, int c) {
19     w[idx] = c, e[idx] = b, ne[idx] = h[a], h[a] = idx++;
20 }
21
22 void query() {
23     int c, st, ed; cin >> c >> st >> ed; //dist[x][oil]表示当前位于点x, 油量为oil的最小花
费
24     vector<vector<int>>> dist(n+2, vector<int>(c+2, 0x3f3f3f3f)), vis(n+2, vector<int>
(c+2));
25     priority_queue<node> pq;
26     pq.push({st, 0, 0});
27     dist[st][0] = 0;
28     while(pq.size()) {
29         auto [x, oil, cost] = pq.top();
30         pq.pop();
31         if(x == ed) { cout << cost << '\n'; return; }
32         if(oil < c) { //诺油量没满, 则可以选择原地不动加油
33             if(dist[x][oil+1] > dist[x][oil] + p[x]) {
34                 dist[x][oil+1] = dist[x][oil] + p[x];
35                 pq.push({x, oil+1, dist[x][oil+1]});
36             }
37         }
38         for(int i = h[x]; ~i; i = ne[i]) {
39             if(oil < w[i]) continue; //诺oil < w[i], 则可以更新其它点
40             int y = e[i];
41             if(dist[y][oil-w[i]] > dist[x][oil]) {
42                 dist[y][oil-w[i]] = dist[x][oil];
43                 pq.push({y, oil-w[i], dist[y][oil-w[i]]});
44             }
45         }
46     }
47     cout << "impossible\n";
48 }
49
50 int main() {
51     memset(h, -1, sizeof h);
52     cin >> n >> m;
53     for(int i = 0; i < n; i++) cin >> p[i];
54     for(int i = 1; i <= m; i++) {
55         int a, b, c; cin >> a >> b >> c;
56         add(a, b, c); add(b, a, c);
57     }
58     cin >> q;
59     while(q--) query();
60 }

```

双向搜索

诸问题有明确的“初态”和“终态”，可以从两端同时进行BFS或DFS，在中间交汇，组成最终答案。

双向BFS

[P10487 Nightmare II - 洛谷 \(luogu.com.cn\)](#)

给定一个 $N \times M$ 的网格，“M”代表男孩每秒可以走3格，“G”代表女孩每秒可以走1格，“Z”代表鬼每秒拓展2格，“X”代表墙，求在不进入鬼的占领区的前提下，男孩和女孩能否会合，若能则输出最短会合时间。

从起始状态和目标状态分别开始，两边轮流进行，每次各扩展一整层，当两边各自有一个状态发生重复时，就说明这两个搜索过程相遇了，就可以合并出起点到终点的最少步数。

```
1  #include <iostream>
2  #include <queue>
3  #include <vector>
4  using namespace std;
5  const int N = 805;
6  string s[N];
7
8  int dx[] = {-1,1,0,0},dy[] = {0,0,-1,1};
9
10 int sol(){
11     int n,m,time = 0;cin >> n >> m;
12     vector<pair<int,int>>z;
13     vector<vector<int>>vis1(n+5,vector<int>(m+5)),vis2(n+5,vector<int>(m+5));//
    分别表示男孩和女孩访问过点的状态，若同时为1则说明相遇了。
14     queue<pair<int,int>>q1,q2;
15     auto check = [&](int x,int y,int time)->bool{//判断当前点是否合法(未进入鬼区)
16         auto dis = [&](int x1,int y1,int x2,int y2){
17             return abs(x1-x2) + abs(y1-y2);
18         };
19         for(int i = 0;i < 2;i++){
20             if(2*time >= dis(x,y,z[i].first,z[i].second)) return 0;
21         }
22         return 1;
23     };
24     for(int i = 1;i <= n;i++){
25         cin >> s[i];s[i] = ' ' + s[i];
26         for(int j = 1;j <= m;j++){
27             if(s[i][j] == 'M') {q1.push({i,j});vis1[i][j] = 1;}
28             if(s[i][j] == 'G') {q2.push({i,j});vis2[i][j] = 1;}
29             if(s[i][j] == 'Z') z.push_back({i,j});
30         }
31     }
32     while(q1.size() || q2.size()){
```

```

33     time++;
34     for(int t = 1;t <= 3;t++){//q1每次拓展3层
35         int siz = q1.size();
36         while(siz--){
37             auto [x,y] = q1.front();
38             q1.pop();
39             if(!check(x,y,time)) continue;
40             for(int i = 0;i < 4;i++){
41                 int nx = x + dx[i],ny = y + dy[i];
42                 if(nx < 1 || nx > n || ny < 1 || ny > m || s[nx][ny] == 'x'
|| vis1[nx][ny]) continue;
43                 if(vis2[nx][ny]) return time;
44                 vis1[nx][ny] = 1;
45                 q1.push({nx,ny});
46             }
47         }
48     }
49     for(int t = 1;t <= 1;t++){//q2每次拓展一层
50         int siz = q2.size();
51         while(siz--){
52             auto [x,y] = q2.front();
53             q2.pop();
54             if(!check(x,y,time)) continue;
55             for(int i = 0;i < 4;i++){
56                 int nx = x + dx[i],ny = y + dy[i];
57                 if(nx < 1 || nx > n || ny < 1 || ny > m || s[nx][ny] == 'x'
|| vis2[nx][ny]) continue;
58                 if(vis1[nx][ny])return time;
59                 vis2[nx][ny] = 1;
60                 q2.push({nx,ny});
61             }
62         }
63     }
64 }
65 return -1;
66 }
67
68 int main(){
69     int t;cin >> t;
70     while(t--) cout << sol() << '\n';
71 }

```

双向DFS

[P10484 送礼物 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P10484)

给定 N 个物品，每个物品体积为 $a[i]$ ，和一个容量为 M 背包，请问最多能装多少体积的物品？
其中 $N \leq 45, 1 \leq a[i], M \leq 2^{31-1}$

本题使用背包求解的话体积过大，指数型枚举的话复杂度过高 2^{45} 。

我们可以把礼物分成两半，对每一半分别搜索，将其能达到的体积分别存放于两个数组A,B中，对B数组进行排序，然后遍历A数组中的每一个元素 x_1 ，在B数组中二分找到 x_2 ，使得 $x_1+x_2 \leq m$ ，用二者之和更新答案。时间复杂度为 $O(2^{\frac{n}{2}+1})$

```
1  #include <iostream>
2  #include <set>
3  #include <vector>
4  #include <algorithm>
5  using namespace std;
6  int n,m;
7
8  void dfs(vector<int>&v,vector<long long>&w,int id,long long sum){
9      if(sum > m) return;
10     if(id == v.size()){
11         w.emplace_back(sum);
12         return;
13     }
14     dfs(v,w,id+1,sum+v[id]);
15     dfs(v,w,id+1,sum);
16 }
17
18 int main(){
19     cin >> m >> n;
20     vector<int>v1,v2;
21     vector<long long>w1,w2;
22     int mid = (n+1)/2;
23     for(int i = 1;i <= mid;i++){
24         int x;cin >> x;
25         v1.emplace_back(x);
26     }
27     for(int i = mid+1;i <= n;i++){
28         int x;cin >> x;
29         v2.emplace_back(x);
30     }
31     dfs(v1,w1,0,0);
32     dfs(v2,w2,0,0);
33     sort(w2.begin(),w2.end());
34     long long ans = 0;
35     for(auto x1:w1){
36         auto x2 = --upper_bound(w2.begin(),w2.end(),m-x1);
37         ans = max(ans,x1+*x2);
38     }
39     cout << ans;
40 }
```

A*

为了提高搜索效率，我们设计一个“估价函数”，将“当前代价+未来估价”的最小状态作为堆顶进行拓展。

为了保证第一次从堆中取出目标状态时得到的就是最优解，我们设计的估价函数 $f(\text{state})$ 不能大于未来的实际代价 $g(\text{state})$ 。

这种带有估价函数的优先队列BFS就称为A*算法。

[178. 第K短路 - AcWing题库](#)

给定N个点，M条边的有向图，求从起点st到终点ed的第k短路（允许经过重复点或边）。

在优先队列BFS中，当节点x第k次被取出时，当前代价即为第k小代价。考虑A*优化。

1. 我们以点x到终点ed的最短距离作为估价函数 $f(x)$ ，建反图以ed为起点求单源最短路即可求得 $f(x)$ 。
2. 建立二叉堆，存储二元组 $(x, \text{dist} + f(x))$ ，x表示当前节点，dist代表当前实际代价， $f(x)$ 表示预估代价。最初堆中只有 $(\text{st}, 0 + f(\text{st}))$
3. 从堆顶取出 $\text{dist} + f(x)$ 的最小二元组进行拓展，如果节点y被取出的次数尚未达到k次，就把新的二元组 $(y, (\text{dist} + w[i]) + f(y))$ 插入堆中。
4. 重复3操作，直到终点ed被取出了k次，此时的dist即为从st到ed的第k短路。

A*算法的时间复杂度上界仍与优先队列BFS相同 $O(K(N + M)\log(N + M))$ ，不过由于估价函数的作用，图中很多节点访问的次数都远小于k，对于大多数数据，能比较快地求出结果，但能构造出数据，使得算法超时/超空间，正解应为可持久化可并堆做法。

```
1  #include <iostream>
2  #include <cstring>
3  #include <queue>
4  using namespace std;
5  const int N = 1003, M = 10004;
6  int n, m, st, ed, k;
7  int h[N], e[M], ne[M], w[M], idx;
8  int f[N];
9  int vis[N];
10
11 struct edge{
12     int a, b, c;
13 }in[M];
14
15 void add(int a, int b, int c){
16     w[idx] = c, e[idx] = b, ne[idx] = h[a], h[a] = idx++;
17 }
18
19 void dijkstra(){
20     memset(f, 0x3f, sizeof f);
21
22     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
23     pq.push({0, ed});
24     f[ed] = 0;
25     while(pq.size()){
```

```

25     int x = pq.top().second;
26     pq.pop();
27     if(vis[x]) continue;
28     else vis[x] = 1;
29     for(int i = h[x];~i;i = ne[i]){
30         int y = e[i];
31         if(f[y] > f[x] + w[i]){
32             f[y] = f[x] + w[i];
33             pq.push({f[y],y});
34         }
35     }
36 }
37 }
38
39 struct node{
40     int x,dist;
41     bool operator < (const node &e2)const{
42         return dist + f[x] > e2.dist + f[e2.x];
43     }
44 };
45
46 int astar(){
47     memset(vis,0,sizeof vis);
48     priority_queue<node>pq;
49     pq.push({st,0});
50     while(pq.size()){
51         auto [x,dist] = pq.top();
52         pq.pop();
53         vis[x]++;
54         if(vis[ed] == k) return dist;
55
56         for(int i = h[x];~i;i = ne[i]){
57             int y = e[i];
58             if(vis[y] <= k) pq.push({y,dist+w[i]});
59         }
60     }
61     return -1;
62 }
63
64 int main(){
65     memset(h,-1,sizeof h);
66     cin >> n >> m;
67     for(int i = 1;i <= m;i++){
68         auto &[a,b,c] = in[i]; cin >> a >> b >> c;
69         add(b,a,c);
70     }
71     cin >> st >> ed >> k;
72     if(st == ed) k++;
73
74     dijkstra();
75
76     idx = 0;

```

```

77     memset(h,-1,sizeof h);
78     for(int i = 1;i <= m;i++){
79         auto [a,b,c] = in[i];
80         add(a,b,c);
81     }
82
83     cout << astar();
84 }

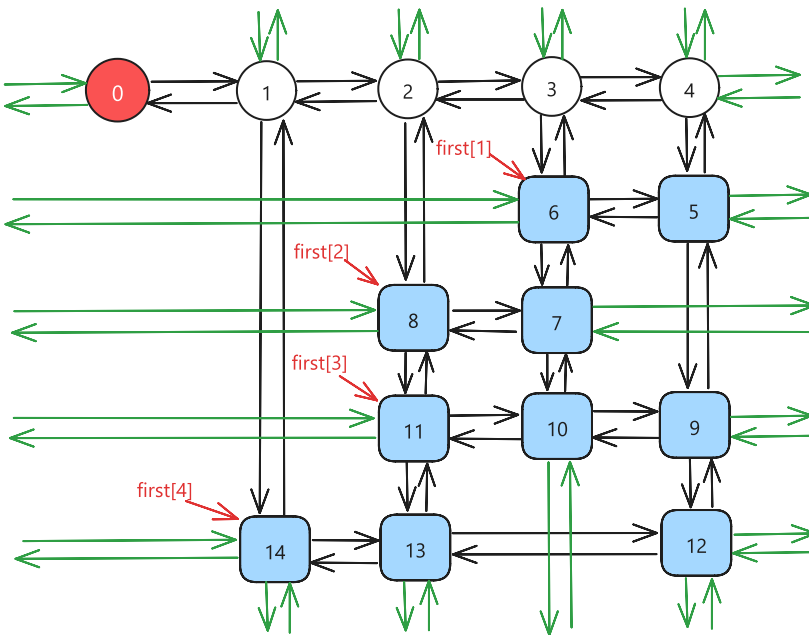
```

IDA*

IDA* 为采用了迭代加深算法的 A* 算法。

Dancing Links

时间复杂度与矩阵中1的个数有关，与矩阵 r ， c 等参数无关，时间复杂度是指数级的 $O(c^n)$ ，其中 c 为某个非常接近1的常数， n 为矩阵中1的个数，尤其擅长处理稀疏矩阵。



精确覆盖

[P4929 【模板】舞蹈链 \(DLX\) - 洛谷 \(luogu.com.cn\)](#)

给定 N 行 M 列的矩阵，每个元素要么是1，要么是0。请从中挑选若干行，使得每列有且仅有一行有1。输出任意一个方案即可，顺序随意。 $N, M \leq 500$ ，保证1的个数不超过5000。

```

1  #include <iostream>
2  #include <vector>
3
4  std::vector<int> res;
5
6  struct DLX{//1_idx

```

```

7     int n,m,idx;
8     std::vector<int> first,siz,stk;//siz[c]存第c列的元素个数,stk存答案,first[r]指向
    第r行的第一个元素
9     std::vector<int> L,R,U,D;//十字循环链表
10    std::vector<int> col,row;//存放第idx个元素的列左边和行坐标
11
12    int ans;
13
14    DLX() {}
15
16    DLX(const int &n, const int &m) {init(n, m);}
17
18    void init(const int &r,const int &c){
19        ans = 1e9;
20        n = r,m = c;
21        first.resize(n+1),siz.resize(m+1);
22        L.resize(m+1),R.resize(m+1),U.resize(m+1),D.resize(m+1);
23        col.resize(m+1),row.resize(m+1);
24        for(int i = 0;i <= m;i++){
25            L[i] = i-1,R[i] = i+1;
26            U[i] = D[i] = i;
27        }
28        L[0] = m,R[m] = 0;
29        idx = m+1;
30    }
31
32    void remove(const int &c){//删除第c列,以及其它与它相关的行
33        L[R[c]] = L[c],R[L[c]] = R[c];
34        for(int i = D[c];i != c;i = D[i]){
35            for(int j = R[i];j != i;j = R[j]){
36                U[D[j]] = U[j];
37                D[U[j]] = D[j];
38                siz[col[j]]--;
39            }
40        }
41    }
42
43    void recover(const int &c){//还原第c列,以及其它与它相关的行,即remove的逆操作
44        for(int i = U[c];i != c;i = U[i]){
45            for(int j = L[i];j != i;j = L[j]){
46                U[D[j]] = D[U[j]] = j;
47                siz[col[j]]++;
48            }
49        }
50        L[R[c]] = R[L[c]] = c;
51    }
52
53    bool dance(){//精确覆盖
54        if(stk.size() >= ans) return 0;//剪枝:如果当前已经超过最优解,直接返回
55        if(!R[0]){//如果0号节点没有右节点,那么矩阵为空,记录答案并返回
56            ans = stk.size(); //当前stk[]即为一组解
57            res = stk;

```

```

58         return 1;
59     }
60     int c = R[0];
61     for(int i = R[0]; i != 0; i = R[i]){
62         if(siz[i] < siz[c]) c = i;
63     }
64     remove(c); //优先选择元素个数最少得一行c,并删除这一列
65     for(int i = D[c]; i != c; i = D[i]){ //遍历这一列其它所有有1的行,递归枚举是否选
择它
66         stk.push_back(row[i]);
67         for(int j = R[i]; j != i; j = R[j]) remove(col[j]);
68         if(dance()) return 1; //任意解
69     // dance(); //最小解
70     for(int j = L[i]; j != i; j = L[j]) recover(col[j]);
71     stk.pop_back();
72     }
73     recover(c);
74     return 0;
75 }
76
77 void remove2(const int &c){ //重复覆盖问题中只需要删除当前列
78     for(int i = D[c]; i != c; i = D[i]){
79         L[R[i]] = L[i]; R[L[i]] = R[i];
80     }
81 }
82
83 void recover2(const int &c){
84     for(int i = U[c]; i != c; i = U[i]){
85         L[R[i]] = R[L[i]] = i;
86     }
87 }
88
89 int f(){ //估价函数
90     int res = 0;
91     std::vector<int> vis(m+1);
92     for(int i = R[0]; i != 0; i = R[i]){
93         if(vis[i]) continue;
94         vis[i] = 1;
95         res++;
96         for(int j = D[i]; j != i; j = D[j]){
97             for(int k = R[j]; k != j; k = R[k]){
98                 vis[col[k]] = 1;
99             }
100         }
101     }
102     return res;
103 }
104
105 bool dance2(){ //重复覆盖
106     if(stk.size() + f() >= ans) return 0;
107     if(!R[0]) {
108         ans = stk.size();

```

```

109         return 1;
110     }
111     int c = R[0];
112     for(int i = R[0]; i != 0; i = R[i]){
113         if(siz[i] < siz[c]) c = i;
114     }
115     for(int i = D[c]; i != c; i = D[i]){
116         for(int j = R[i]; j != i; j = R[j]) remove2(j);
117         remove2(i);
118         stk.push_back(row[i]);
119         // if(dance2()) return 1; 任意解
120         dance2(); //最小解
121         stk.pop_back();
122         recover2(i);
123         for(int j = L[i]; j != i; j = L[j]) recover2(j);
124     }
125     return 0;
126 }
127
128 void insert(const int &r, const int &c){ //在第r行,第c列插入一个节点
129     col.push_back(c), row.push_back(r);
130     siz[c]++;
131     U.push_back(c), D.push_back(D[c]);
132     U[D[c]] = idx; D[c] = idx;
133     if(first[r] == 0){ //如果第r行没有元素,那么直接插入一个元素,并使first[r]指向该元
        素
134         first[r] = idx;
135         L.push_back(idx), R.push_back(idx);
136     }
137     else{ //否则把idx插入到c的正下方,把idx插入到first(r)的正右方
138         R.push_back(R[first[r]]);
139         L[R[first[r]]] = idx;
140         L.push_back(first[r]);
141         R[first[r]] = idx;
142     }
143     idx++;
144 }
145 };
146
147
148 int main() {
149     int n, m; std::cin >> n >> m;
150     DLX dlx(n, m);
151     for(int i = 1; i <= n; i++){
152         for(int j = 1; j <= m; j++){
153             int x; std::cin >> x;
154             if(x) dlx.insert(i, j);
155         }
156     }
157     dlx.dance();
158     if(dlx.ans == 1e9) {
159         std::cout << "No Solution!";

```

```

160     }
161     else{
162         for(auto x:res){
163             std::cout << x << ' ';
164         }
165     }
166 }

```

P1784 数独 - 洛谷 (luogu.com.cn)

求解大小为 $w*w$ 的数独

```

1  #include <iostream>
2  #include <vector>
3
4  const int N = 10;
5  int w = 9;
6  int sw = 3;//sqrt(w)
7  int a[N][N];
8
9  int main(){
10     DLX dlx;
11     dlx.init(w*w*w,w*w*4);
12     for(int i = 0;i < w;i++){
13         for(int j = 0;j < w;j++){
14             std::cin >> a[i][j];
15             for(int k = 1;k <= w;k++){
16                 if(a[i][j] != 0 && a[i][j] != k) continue;
17                 int id = i*w*w + j*w + k;
18                 dlx.insert(id,i*w + j + 1);
19                 dlx.insert(id,i*w + w*w + k);
20                 dlx.insert(id,j*w + w*w*2 + k);
21                 dlx.insert(id,w*w*3 + (i/sw*sw + j/sw)*w + k);
22             }
23         }
24     }
25
26     dlx.dance();
27
28     for(int i = 0;i < w;i++){
29         for(int j = 0;j < w;j++){
30             std::cout << a[i][j] << ' ';
31         }
32         std::cout << '\n';
33     }
34 }

```


重复覆盖

remove、recover、dance操作略有不同。

调用dlx.dance2()即可。

因为重复覆盖问题方案一般比精确覆盖多，搜索基于IDA*优化，加入了估价函数f()。

[Luogu 蓝桥杯2019省A 糖果](#)

倍增

倍增法（英语：binary lifting），顾名思义就是翻倍。它能够使线性的处理转化为对数级的处理，大大地优化时间复杂度。

这个方法在很多算法中均有应用，其中最常用的是 RMQ 问题和求 LCA(最近公共祖先)。

ST表

ST表(Sparse Table,稀疏表)基于倍增思想,用于解决[可重复贡献问题][区间重合区域不影响结果，如「RMQ」、「区间按位与」、「区间按位或」、「区间 GCD」]的数据结构，**不支持修改**

$O(N \log N)$ 预处理 $O(1)$ 查询

max = 14							
max = 14							
max = 13							
max = 0							
0	13	14	4	13	1	5	7

令 $st(i, j)$ 表示区间 $[i, i + 2^j - 1]$ 的最大值，显然 $st(i, 0) = a_i$

根据定义式，第二维就相当于倍增的时候「跳了 $2^j - 1$ 步」，依据倍增的思路，写出状态转移方程：

$$st(i, j) = \max(st(i, j-1), st(i + 2^{j-1}, j-1))$$

对于每个询问 $[l, r]$ ，我们把它分成两部分： $[l, l + 2^s - 1]$ 与 $[r - 2^s + 1, r]$ ，其中

$s = \lfloor \log_2(r - l + 1) \rfloor$ 。两部分的结果的最大值就是回答。

```
1 //https://www.luogu.com.cn/problem/P2880
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 namespace S_T { //1_idx
6     const int N = 1000006;
7     int lg2[N];
8 }
```

```

9      struct info{
10          int mx,mn; //mn,gcd,and,or;
11
12          info(){}
13          info(const int &x) { //init_info
14              mx = mn = x;
15          }
16
17          info friend operator + (const info &e1,const info &e2) { //updaet_info
18              info ans;
19              ans.mx = std::max(e1.mx,e2.mx);
20              ans.mn = std::min(e1.mn,e2.mn);
21              return ans;
22          }
23      };
24
25      void init_lg2(){ //下取整
26          lg2[0] = -1;
27          for(int i = 1;i < N;i++){
28              lg2[i] = lg2[i>>1]+1;
29          }
30      }
31
32      template<typename T>
33      struct ST{
34          int n,m;
35          std::vector<std::vector<info>>>st;
36
37          ST(){}
38          ST(const T &v) { //1_idx
39              if(~lg2[0]) init_lg2();
40              n = v.size()-1,m = lg2[v.size()];
41              st = std::vector<std::vector<info>>>(n+1,std::vector<info>(m+1));
42
43              for(int i = 1;i < v.size();i++){ st[i][0] = v[i]; }
44              for(int j = 1;(1<<j) < v.size();j++){
45                  int pj = 1 << (j-1);
46                  for(int i = 1;i+(1<<j)-1 < v.size();i++){
47                      st[i][j] = st[i][j-1] + st[i+pj][j-1];
48                  }
49              }
50          }
51
52          info query(int l,int r) {
53              int x = lg2[r-l+1];
54              info ans = st[l][x] + st[r-(1<<x)+1][x];
55              return ans;
56          }
57      };
58  }
59  using S_T::ST,S_T::info;
60

```

```

61 int main() {
62     int n,q; std::cin >> n >> q;
63     vector<int> a(n+1);
64     for(int i = 1;i <= n;i++){
65         std::cin >> a[i];
66     }
67     ST t(a);
68     while(q--){
69         int l,r; std::cin >> l >> r;
70         auto ans = t.query(l,r);
71         std::cout << ans.mx - ans.mn << '\n';
72     }
73 }

```

二维ST表

[Check Corners - HDU 2888 - Virtual Judge](#)

查询子矩阵RMQ

```

1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  namespace S_T { //1_idx
6      const int N = 355, M = 9;
7      int lg2[N];
8
9      struct info {
10         int mx; //gcd,and,or;
11
12         info() {}
13         info(const int &x) { //init_info
14             mx = x;
15         }
16
17         info friend operator + (const info &e1, const info &e2) { //update_info
18             info ans;
19             ans.mx = std::max(e1.mx, e2.mx);
20             return ans;
21         }
22     } st[N][N][M][M]; //vector内存开销较大,尽量使用定长数组
23
24     void init_lg2() {
25         lg2[0] = -1;
26         for(int i = 1; i < N; i++) {
27             lg2[i] = lg2[i>>1] + 1;
28         }
29     }

```

```

30
31     template<typename T>
32     struct ST{
33         int n,m;
34
35         ST(){}
36         ST(const std::vector<std::vector<T>> &mat) { //1_idx
37             if(~lg2[0]) init_lg2();
38             n = mat.size()-1;
39             m = mat[0].size()-1;
40
41             for (int ni = 0; 1 << ni <= n; ni++) {
42                 for (int nj = 0; 1 << nj <= m; nj++) {
43                     for (int i = 1; i + (1 << ni) - 1 <= n; i++) {
44                         for (int j = 1; j + (1 << nj) - 1 <= m; j++) {
45                             if (!ni && !nj) st[i][j][0][0] = info(mat[i][j]);
46                             if (ni) st[i][j][ni][nj] = st[i][j][ni - 1][nj] +
st[i + (1 << ni - 1)][j][ni - 1][nj];
47                             if (nj) st[i][j][ni][nj] = st[i][j][ni][nj - 1] +
st[i][j + (1 << nj - 1)][ni][nj - 1];
48                         }
49                     }
50                 }
51             }
52         }
53
54         info query(int x1, int y1, int x2, int y2) {
55             int kx = lg2[x2 - x1 + 1];
56             int ky = lg2[y2 - y1 + 1];
57
58             int len_x = 1 << kx;
59             int len_y = 1 << ky;
60
61             info res1 = st[x1][y1][kx][ky];
62             info res2 = st[x2 - len_x + 1][y1][kx][ky];
63             info res3 = st[x1][y2 - len_y + 1][kx][ky];
64             info res4 = st[x2 - len_x + 1][y2 - len_y + 1][kx][ky];
65
66             return res1 + res2 + res3 + res4;
67         }
68     };
69 }
70 using S_T::ST, S_T::info;
71
72 int n,m;
73
74 void sol() {
75     std::vector<std::vector<int>> a(n + 1, std::vector<int>(m + 1));
76     for (int i = 1; i <= n; i++) {
77         for (int j = 1; j <= m; j++) {
78             std::cin >> a[i][j];
79         }

```

```

80     }
81
82     ST t(a);
83     int q; std::cin >> q;
84     while (q--) {
85         int x1, y1, x2, y2; std::cin >> x1 >> y1 >> x2 >> y2;
86         int mx = t.query(x1, y1, x2, y2).mx;
87         std::cout << mx << ' ';
88         if (mx == std::max({a[x1][y1], a[x1][y2], a[x2][y1], a[x2][y2]}))
89             std::cout << "yes\n";
90         else std::cout << "no\n";
91     }
92 }
93
94 int main() {
95     std::ios::sync_with_stdio(false); std::cin.tie(0);
96     while (std::cin >> n >> m) sol();
97 }

```

贪心

[贪心 - OI Wiki \(oi-wiki.org\)](https://oi-wiki.org/)

区间问题

区间选点

给定 N 个闭区间 $[a_i, b_i]$ ，请在数轴上选择尽量少的点，使得每个区间内至少包含一个选出的点。输出选择的点的最小数量。（位于区间端点上的点也算作区间内）

```

1 //https://www.acwing.com/problem/content/907/
2 #include <iostream>
3 #include <algorithm>
4 using namespace std;
5 const int N = 100005;
6 int n;
7
8 struct Edge{
9     int l,r;
10 }e[N];
11
12 int main(){
13     cin >> n;
14     for(int i = 1; i <= n; i++){ cin >> e[i].l >> e[i].r; }
15
16     sort(e+1, e+n+1, [](auto&e1, auto&e2){return e1.l < e2.l;}); //按左端点从小到大排序
17
18     int ans = 1;

```

```

19     int ed = e[1].r;
20
21     for(int i = 2; i <= n; i++){
22         if(e[i].l > ed){
23             ans++;
24             ed = e[i].r;
25         }
26         else{
27             ed = min(ed, e[i].r);
28         }
29     }
30     cout << ans;
31 }

```

```

1 //畜栏预定 https://www.acwing.com/problem/content/113/
2 //给定N段区间[l,r],当两个区间相交时(包括端点),这两头牛不能安排在同一个畜栏吃草
3 //求需要最小畜栏数目和每头牛对应的畜栏方案
4 #include <iostream>
5 #include <algorithm>
6 #include <queue>
7 using namespace std;
8 const int N = 100005;
9 using pii = pair<int, int>;
10 int n;
11 int id[N];
12
13 struct Edge{
14     int l, r, rk;
15 }e[N];
16
17 int main(){
18     cin >> n;
19     for(int i = 1; i <= n; i++){
20         cin >> e[i].l >> e[i].r;
21         e[i].rk = i;
22     }
23
24     sort(e+1, e+n+1, [](auto &e1, auto &e2){return e1.l < e2.l;});
25     //按左端点将区间排序
26
27     priority_queue<pii, vector<pii>, greater<pii>> pq;
28     //堆中存放的元素为<当前分组内区间右端点的最小值, 畜栏编号>
29
30     for(int i = 1; i <= n; i++){
31         if(pq.empty() || pq.top().first >= e[i].l){//新建堆
32             //当前堆为空 或 堆顶最小值右端点r与当前牛左端点l重合
33             pii t = {e[i].r, pq.size()};
34             id[e[i].rk] = t.second;
35             pq.push(t);
36         }
37         else{//使用原来的堆

```

```

38         auto t = pq.top();
39         pq.pop();//弹出堆顶
40         t.first = e[i].r;//让当前牛使用该畜栏
41         id[e[i].rk] = t.second;//记录答案
42         pq.push(t);
43     }
44 }
45 cout << pq.size() << endl;
46 for(int i = 1; i <= n; i++){
47     cout << id[i]+1 << endl;
48 }
49 }

```

区间合并

现给定 n 个闭区间 a_i, b_i ($1 \leq i \leq n$)。这些区间的并可以表示为一些不相交的闭区间的并。在这些表示方式中找出包含最少区间的方案。

```

1 //https://www.luogu.com.cn/problem/P2434
2 #include <bits/stdc++.h>
3 using PII = std::pair<int, int>;
4 using namespace std;
5
6 struct P {
7     int l, r;
8 }p[50005];
9
10 queue<PII>q;
11
12 bool cmp(P p1, P p2) {
13     return p1.l < p2.l;
14 }
15
16 int main() {
17     int n; cin >> n;
18     for (int i = 0; i < n; i++) {
19         cin >> p[i].l >> p[i].r;
20     }
21
22     sort(p, p + n, cmp);
23
24     int op = p[0].l, ed = p[0].r;
25     for (int i = 0; i < n; i++) {
26         if (ed >= p[i].l) {
27             ed = max(ed, p[i].r);
28         }
29         else {
30             q.push({ op, ed });
31             op = p[i].l;
32             ed = p[i].r;

```

```

33     }
34 }
35 q.push({ op,ed });
36
37 while (q.size()) {
38     cout << q.front().first << ' ' << q.front().second << endl;
39     q.pop();
40 }
41 }

```

不相交区间

给定 N 个闭区间 $[a_i, b_i]$ ，请在数轴上选择若干区间，使得选中的区间之间互不相交（包括端点）。输出可选取区间的最大数量。

```

1 //https://www.acwing.com/problem/content/910/
2 #include <iostream>
3 #include <algorithm>
4 using namespace std;
5 const int N = 100005;
6 int n;
7 struct Point {
8     int x, y;
9 }p[N];
10
11 bool cmp(Point p1, Point p2) {
12     return p1.x < p2.x; //以左端点升序排序
13 }
14
15 int main() {
16     cin >> n;
17     for (int i = 1; i <= n; i++) {
18         cin >> p[i].x >> p[i].y;
19     }
20
21     sort(p + 1, p + n + 1, cmp);
22
23     int ans = 1;
24     int r = p[1].y;
25     for (int i = 2; i <= n; i++) {
26         if (p[i].y < r) {
27             r = p[i].y;
28         }
29         if (p[i].x > r) {
30             ans++;
31             r = p[i].y;
32         }
33     }

```



```
34     cout << ans;
35 }
```

区间覆盖

给定 N 个闭区间 $[a_i, b_i]$ 以及一个线段区间 $[s, t]$ ，请你选择尽量少的区间，将指定线段区间完全覆盖。输出最少区间数，如果无法完全覆盖则输出 -1 。

核心思想：按左端点排序，在左端点 l 都小于等于 a 的情况下，取右端点最大的小区间。时间复杂度 $O(N \log N) + O(N)$ 。

注意：本题为线段区间（即连续区间），若要求覆盖离散区间，只需要第41行将a更新为r+1。

```
1 //https://www.acwing.com/problem/content/description/909/
2 #include <iostream>
3 #include <algorithm>
4 using namespace std;
5 const int N = 100005, INF = 0x3f3f3f3f;
6
7 struct P {
8     int x, y;
9 } p[N];
10
11 bool cmp(P p1, P p2) {
12     return p1.x < p2.x;
13 }
14
15 int main() {
16     int a, b; cin >> a >> b;
17     int n; cin >> n;
18     for (int i = 1; i <= n; i++) {
19         cin >> p[i].x >> p[i].y;
20     }
21
22     sort(p + 1, p + n + 1, cmp);
23     int ans = 0;
24     bool flag = 0;
25     for (int i = 1; i <= n; i++) {
26         int j = i, r = -INF;
27         while (j <= n && p[j].x <= a) { //所有左端点都小于等于a的区间中,取右端点最大的
28             r = max(r, p[j].y);
29             j++;
30         }
31
32         if (r < a) {
33             ans = -1;
34             break;
35         }
```

```

36         ans++;
37         if (r >= b) {
38             flag = 1;
39             break;
40         }
41         a = r;
42         i = j - 1;
43     }
44     if (!flag) ans = -1;
45     cout << ans;
46     return 0;
47 }

```

区间分组

(某个点) 最多同时重叠区间个数

给定 N 个闭区间 $[a_i, b_i]$ ，请你将这些区间分成若干组，使得每组内部的区间两两之间（包括端点）没有交集，并使得组数尽可能小。输出最小组数。

我们可以把所有开始时间和结束时间排序，遇到开始时间就把需要的教室数 cnt 加1，遇到结束时间就把需要的教室数 cnt 减1，在一系列需要的教室个数 cnt 变化的过程中， cnt 的峰值就是多同时进行的活动数，也是我们至少需要的教室数。

如果值域较小，可以写差分

```

1 //https://www.acwing.com/problem/content/908/
2 #include <iostream>
3 #include <algorithm>
4 #include <vector>
5 #define fastio ios::sync_with_stdio(0),cin.tie(0)
6 using namespace std;
7 const int N = 100005;
8 vector<pair<int,int>>v;
9
10 struct Edge{
11     int l,r;
12 }e[N];
13
14 int main(){
15     fastio;
16     int n;cin >> n;
17     for(int i = 1;i <= n;i++){
18         auto&[l,r] = e[i];
19         cin >> l >> r;
20         v.emplace_back(l,0);//0代表开始
21         v.emplace_back(r,1);//1代表结束
22     }
23     sort(v.begin(),v.end());

```

```

24     int ans = 0;
25     int cnt = 0;
26     for(int i = 0; i < v.size(); i++){
27         if(v[i].second == 0) cnt++;
28         else cnt--;
29         ans = max(ans, cnt);
30     }
31     cout << ans;
32 }

```

(任意长度为d的区间) 最多/最少包含不同区间个数 (重叠区间的长短并不重要)

```

1 //https://codeforces.com/contest/2014/problem/D
2 #include <bits/stdc++.h>
3 #define INF 0x3f3f3f3f
4 using ll = long long;
5 using namespace std;
6
7 void sol(){
8     ll n, d, k; cin >> n >> d >> k;
9     vector<ll> a(n+5);
10    for(int i = 1; i <= k; i++){ //差分, 让区间[l-d+1, r]加1
11        int l, r; cin >> l >> r;
12        a[max(l-d+1, (ll)0)]++;
13        a[r+1]--;
14    }
15    for(int i = 1; i <= n; i++){ a[i] += a[i-1]; } //a[i]表示从点i开始, 长为d的一段区间,
    包含不同区间的个数
16    ll kb = 0, pb = 0, km = INF, pm = 0;
17    for(int i = 1; i <= n-d+1; i++){
18        if(a[i] > kb){ kb = a[i]; pb = i; }
19        if(a[i] < km){ km = a[i]; pm = i; }
20    }
21    cout << pb << ' ' << pm << endl;
22 }
23
24 int main() {
25     int T = 1; cin >> T;
26     while(T--){ sol(); }
27 }

```

区间包含

```

1 //https://acm.hdu.edu.cn/showproblem.php?pid=7497
2 #include <bits/stdc++.h>
3 using ll = long long;
4 using namespace std;

```

```

5  int n,m;
6
7  struct Edge{
8      ll l,r;
9      ll len;
10 };
11
12 bool cmp(ll x,ll y){
13     return x < y;
14 }
15
16 bool cmp1(pair<ll,ll>x,pair<ll,ll>y){
17     if(x.first != y.first) return x.first < y.first;
18     else{ return x.second < y.second; }
19 }
20
21 void sol(){
22     cin >> n >> m;
23     vector<Edge>a(n),b(m);
24     vector<ll>all;
25     for(int i = 0;i < n;i++){
26         cin >> a[i].l >> a[i].r;
27         all.emplace_back(a[i].l*2+1);
28         all.emplace_back(a[i].r*2);
29     }
30     for(int i = 0;i < m;i++){
31         cin >> b[i].l >> b[i].r;
32         b[i].len = 2*(b[i].r-b[i].l);
33         all.emplace_back(b[i].l*2+1);
34         all.emplace_back(b[i].r*2);
35     }
36     sort(all.begin(),all.end(),cmp);
37     int cnt = 0;
38     for(int i = 0;i < all.size();i++){//判断A组区间与B组区间中是否存在区间香蕉(区间分
组)
39         if(all[i]&1){ cnt++; }
40         else { cnt--; }
41         if(cnt > 1){ cout << "No" << endl; return; }
42     }
43
44     vector<pair<ll,ll>>a1;
45     for(int i = 0;i < n;i++){
46         a1.emplace_back(a[i].l,2);
47         a1.emplace_back(a[i].r,3);
48     }
49     for(int i = 0;i < m;i++){
50         a1.emplace_back(b[i].r,0);
51         a1.emplace_back(b[i].r+b[i].len,4);
52     }
53     sort(a1.begin(),a1.end(),cmp1);
54
55     int ok = 0,cla = 0;//判断A组区间是否完全包含于C组区间(区间分组加强版)

```

```

56     for(int i = 0;i < a1.size();i++){
57         if(a1[i].second == 0) ok++;//0代表清醒状态开始
58         if(a1[i].second == 4) ok--;//4代表清醒状态结束
59         if(a1[i].second == 2) cla++;//2代表上课开始
60         if(a1[i].second == 3) cla--;//3代表上课结束
61         if(cla >= 1 && ok <= 0){//如果出现当前正在上课，并且状态不清醒，返回No
62             cout << "No" << endl; return;
63         }
64     }
65     cout << "Yes" << endl;
66 }
67
68 int main() {
69     int T = 1; cin >> T;
70     while(T--){sol();}
71 }

```

绝对值不等式

货仓选址

在一条数轴上有 N 家商店，它们的坐标分别为 $A_1 \sim A_N$ 。

现在需要在数轴上(任意一点)建立一家货仓，每天清晨，从货仓到每家商店都要运送一车商品。

为了提高效率，求把货仓建在何处，可以使得货仓到每家商店的距离之和最小。

在中位数处建点可以使得答案最小

```

1 //https://www.acwing.com/problem/content/106/
2 #include <iostream>
3 #include <cmath>
4 #include <algorithm>
5 using namespace std;
6 const int N = 100005;
7 int n,a[N];
8
9 int main(){
10     cin >> n;
11     for(int i = 1;i <= n;i++){
12         cin >> a[i];
13     }
14     sort(a+1,a+n+1);
15     long long ans = 0;
16     for(int i = 1;i <= n;i++){
17         ans+=abs(a[i]-a[n+1 >> 1]);
18     }
19     cout << ans;
20 }

```

均分纸牌

n个人坐在一排,每人有a[i]张纸牌,每次可将任意数量纸牌给相邻的一个人,求使所有人纸牌数相等的最小操作次数

```
1 //https://www.acwing.com/problem/content/1538/
2 #include <iostream>
3 using namespace std;
4 const int N = 10004;
5 int a[N];
6 int ans, sum;
7
8 int main(){
9     int n; cin >> n;
10    for(int i = 1; i <= n; i++){
11        cin >> a[i];
12        sum += a[i];
13    }
14    int avg = sum/n;
15
16    for(int i = 1; i < n; i++) {
17        a[i] -= avg; //最终所有人的纸牌一定变为平均数
18        if(a[i]){ //不够的从i+1取, 多的给i+1
19            a[i+1] += a[i];
20            ans++;
21        }
22    }
23    cout << ans; //每次可移动任意张, 最小次数
24 }
```

环形均分纸牌

n个人围在一圈,每人有a[i]张纸牌,每次可将1张纸牌给相邻的一个人,求使所有人纸牌数相等的最小操作次数

```
1 //https://www.luogu.com.cn/problem/P2512
2 #include <iostream>
3 #include <algorithm>
4 using namespace std;
5 using ll = long long;
6 const int N = 1000006;
7 ll a[N], s[N];
8 int n;
9 ll sum, ans;
10
11 int main(){
12     cin >> n;
13     for(int i = 1; i <= n; i++) cin >> a[i], sum += a[i];
```

```

14
15     ll avg = sum/n;
16
17     for(int i = 1; i <= n; i++){
18         s[i] = s[i-1] + a[i] - avg;
19     }
20     //s为a的所差的前缀和数组
21     //选取s的中位数最优，问题变为对s数组的货仓选址问题
22
23     sort(s+1, s+n+1);
24
25     ll mid = s[(n+1)/2];
26
27     for(int i = 1; i <= n; i++){
28         ans += abs(s[i] - mid);
29     }
30     cout << ans; //每次移动一张，最小次数
31 }

```

排序不等式

邻项交换法 证明在任意局面下，任何对局部最优策略的微小改变都会造成整体结果变差。经常用于以“排序”为贪心策略的证明。

诺交换相邻两项不会影响其他项的值，单独分析这两项，找出排序策略cmp

耍杂技的牛

奶牛们站在彼此的身上，形成一个高高的垂直堆叠。

这 N 头奶牛中的每一头都有着自己的重量 w_i 以及自己的强壮程度 s_i 。

一头牛支撑不住的可能性取决于它头上所有牛的总重量（不包括它自己）减去它的身体强壮程度的值，现在称该数值为风险值，风险值越大，这只牛撑不住的可能性越高。

确定奶牛的排序，使得所有奶牛的风险值中的最大值尽可能的小，求出该值。

将所有牛按 $w+s$ 值从小到大排序时，最大的风险值一定最小

相邻两牛交换不会影响其它牛的风险值，所以只需要分析这两头牛即可

风险值	i牛	i+1牛
交换前:	$w[1]+...+w[i-1]-s[i]$	$w[1]+...+w[i-1]+w[i]-s[i+1]$
交换后:	$w[1]+...+w[i-1]-s[i+1]$	$w[1]+...+w[i-1]+w[i+1]-s[i]$
交换前(化简):	$s[i+1]$	$w[i]+s[i]$

风险值	i牛	i+1牛
交换后(化简):	s[i]	w[i+1]+s[i+1]

i牛风险值 -= s, i+1牛风险值 += s+w

```

1 //https://www.acwing.com/problem/content/127/
2 #include <iostream>
3 #include <algorithm>
4 using namespace std;
5 const int N = 50004;
6 int n;
7 struct S{
8     int sum,w,s;
9 }s[N];
10 long long suf[N];
11
12 bool cmp(S x,S y){
13     return x.sum < y.sum;
14 }
15
16 int main(){
17     cin >> n;
18     for(int i = 1;i <= n;i++){
19         int a,b;cin >> a >> b;
20         s[i] = {a+b,a,b};
21     }
22     sort(s+1,s+n+1,cmp);
23     long long ans = -0x3f3f3f3f;
24     for(int i = 1;i <= n;i++){
25         suf[i] = suf[i-1]+s[i].w;
26         ans = max(ans,suf[i-1]-s[i].s);
27     }
28     cout << ans;
29 }

```

```

1 //栈压缩: https://qoj.ac/problem/9379
2 //国王游戏: https://www.acwing.com/problem/content/description/116/

```


后悔解法

无论当前的选项是否最优都接受，然后进行比较，如果选择之后不是最优了，则反悔，舍弃掉这个选项；否则，正式接受。如此往复。

[Luogu P2949 工作调度](#)

给定 N 个工作，每个工作截止日期为 D_i ，如果能在截止日期前完成则会获得 P_i 的报酬，每天只能完成一项工作。 $1 \leq N \leq 10^5$, $1 \leq D_i, P_i \leq 10^9$

```
1 //https://www.luogu.com.cn/problem/P2949
2 #include <iostream>
3 #include <queue>
4 #include <algorithm>
5 using namespace std;
6 using ll = long long;
7 using pii = pair<ll,ll>;
8 const int N = 100005;
9 ll ans;
10
11 struct Edge{
12     ll d,p;
13 }e[N];
14
15
16 int main(){
17     int n;cin >> n;
18     for(int i = 1; i <= n;i++){ cin >> e[i].d >> e[i].p; }
19
20     sort(e+1,e+n+1,[](auto &e1,auto &e2){return e1.d < e2.d;});
21     //将每一项工作按截止时间从小到大排序
22
23     priority_queue<ll,vector<ll>,greater<ll>>pq; //小根堆存决定做的工作的报酬p
24                                           //pq.size()即为做这些工作的最少安排时间
25     for(int i = 1;i <= n;i++){
26         if(e[i].d <= pq.size()){ //如果当前工作截止时间与已经安排的时间冲突
27             if(e[i].p > pq.top()){ //且选择当前工作比之前决定的工作报酬更高,则反悔之前报
28                 //ans += e[i].p - pq.top();
29                 pq.pop();
30                 pq.push(e[i].p);
31             }
32         }
33         else{ //否则安排当前决定
34             //ans += e[i].p;
35             pq.push(e[i].p);
36         }
37     }
38     while(pq.size()){
39         ans += pq.top();
```

```

40     pq.pop();
41 }
42 cout << ans;
43 }

```

```

1 //https://codeforces.com/problemset/problem/1526/C2
2 //给你一个长度为n的序列A[]，要求你找出最长的一个子序列使得这个子序列任意前缀和都非负。
3 #include <bits/stdc++.h>
4 int T = 1;
5 using ll = long long;
6 using namespace std;
7
8 void sol(){
9     int n;cin >>n;
10    unsigned ans = 0;
11    ll sum = 0;
12    priority_queue<ll,vector<ll>,greater<ll>>pq;
13    while(n--){
14        int x;cin >> x;
15        pq.push(x);//无论当前选择是否最优，先接受
16        sum += x;
17        while(sum < 0){//若接受后不是最优，则反悔之前最差的决定
18            sum -= pq.top();
19            pq.pop();
20        }
21        ans = max(ans,pq.size());
22    }
23    cout << ans << endl;
24 }
25
26 int main() {
27     while(T--){ sol(); }
28 }

```

根号分治

根号分治，是一种对数据进行点分治的分治方式，它的作用是优化暴力算法，类似与分块，但应用范围比分块更广。

具体来说，对于所进行的操作，按照某个点 B 划分，分为大于 B 以及小于 B 两个部分，两部分使用不同的方式处理。（一般以根号为分界 $B = \sqrt{N}$ ，这样复杂度最平衡）。将两个暴力算法“拼接在一起”，实现优化复杂度的作用。

[Colorful Graph \(★6\) - AtCoder typical90 ce - Virtual Judge \(vjudge.net\)](#)

给定N个点M条边的无向图，初始时每个点颜色为1。再给定Q次查询，每次查询给定两个整数{x,c}，输出当前点x的颜色，然后将点x及其所有相邻的点颜色改为c。 $1 \leq N, M, Q \leq 2e5$

`ans[x]={time,color}` 表示当前点最后被更新时的时间戳和颜色， `flag[x]={time,color}` flag状态标记，表示当前节点在time时更新应周围节点为color。

以度数是否大于 $\sqrt{N}$ 为分界，将点划分为重点和轻点。轻点直接枚举，重点打标记。

查询当前点：对于轻点，直接枚举所有邻接点更新自身。对于重点，本身已经被其它点更新。

更新邻接点：只需要更新周围的重点。

```
1  #include <bits/stdc++.h>
2
3  int main(){
4      int n,m;std::cin >> n >> m;
5      int sn = sqrt(m<<1);
6
7      std::vector<std::vector<int>>>e(n+1);
8      std::vector<int>du(n+1);
9      for(int i = 1;i <= m;i++){
10         int x,y;std::cin >> x >> y;
11         e[x].emplace_back(y);
12         e[y].emplace_back(x);
13         du[x]++; du[y]++;
14     }
15
16     for(int i = 1;i <= n;i++){
17         std::sort(e[i].begin(),e[i].end(), [&](int x,int y){return du[x] >
du[y];});
18     }
19
20     std::vector<std::pair<int,int>>ans(n+1,{0,1}),flag(n+1,{0,1});
21
22     int q;std::cin >> q;
23     for(int i = 1;i <= q;i++){
24         int x,c;std::cin >> x >> c;
25         if(du[x] <= sn){
26             for(auto& y:e[x]){
27                 ans[x] = std::max(ans[x],flag[y]);
28             }
29         }
30         std::cout << ans[x].second << '\n';
31         ans[x] = flag[x] = {i,c};
32         for(auto& y:e[x]){
33             if(du[y] <= sn) break;
34             ans[y] = flag[x];
35         }
36     }
37 }
```

离线算法

莫队

普通莫队

对于序列上的区间询问问题，如果从 $[l, r]$ 的答案能够 $O(1)$ 扩展到 $[l-1, r], [l+1, r], [l, r+1], [l, r-1]$ 的答案，那么可以在 $O(N\sqrt{N})$ 的时间复杂度内求出所有询问的答案。

	块长建议	时间复杂度
普通莫队	$\text{sqrt}(n)$ 或 $n/\text{sqrt}(m)$	$O(N\sqrt{N})$
带修莫队	$\text{pow}(n, 2.0/3.0)$	$O(N^{\frac{5}{3}})$
回滚莫队	$\text{sqrt}(n)$	$O(N\sqrt{N})$
树上莫队	$\text{sqrt}(2n)$ 或 $\text{sqrt}(2n)/\text{sqrt}(m)$	

[SP3267 DQUERY - D-query - 洛谷](#)

求区间有多少个不同数字

```
1  #include <iostream>
2  #include <algorithm>
3  #include <cmath>
4
5  const int N = 1000006;
6  int n, q, siz;
7  int a[N]; //如果值域过大,则需要离散化
8
9  struct Ask {
10     int l, r, id;
11     bool operator < (const Ask &e2) const { //奇偶排序优化
12         if ((l - 1) / siz != (e2.l - 1) / siz) return l < e2.l; //优先按块号排序
13         if (((l - 1) / siz) & 1) return r < e2.r; //同一块号,按右端点排序
14         return r > e2.r;
15     }
16 } ask[N];
17
18 int cnt[N]; //cnt[x] 记录当前区间[l,r],x出现的次数 注意值域大小
19 int ans[N]; //ans[id] = sum 记录第id个区间的答案
20 int sum; //记录不同数的个数
```

```

21
22 void add(int x) {
23     if (++cnt[x] == 1) {
24         sum++;
25     }
26 }
27
28 void del(int x) {
29     if (--cnt[x] == 0) {
30         sum--;
31     }
32 }
33
34 void init() {
35     siz = std::sqrt(n);
36     std::sort(ask + 1, ask + q + 1);
37     for (int i = 1, l = 1, r = 0; i <= q; i++) { //注意顺序不能随意改变
38         while (l > ask[i].l) add(a[--l]);
39         while (r < ask[i].r) add(a[++r]);
40         while (l < ask[i].l) del(a[l++]);
41         while (r > ask[i].r) del(a[r--]);
42         ans[ask[i].id] = sum;
43     }
44 }
45
46 int main() {
47     std::cin >> n;
48     for (int i = 1; i <= n; i++) {
49         std::cin >> a[i];
50     }
51
52     std::cin >> q;
53     for (int i = 1; i <= q; i++) {
54         std::cin >> ask[i].l >> ask[i].r;
55         ask[i].id = i;
56     }
57
58     init();
59
60     for (int i = 1; i <= q; i++) {
61         std::cout << ans[i] << '\n';
62     }
63 }

```

带修莫队

[P1903 [国家集训队](#)] 数颜色 / 维护队列 - 洛谷

单点修改，求区间有多少个不同数字

在普通莫队的基础上增加一维时间戳，记录修改的时间。

```
1  #include <iostream>
2  #include <algorithm>
3  #include <cmath>
4
5  const int N = 1000006;
6  int n, q, siz;
7  int a[N];
8
9  struct Ask {
10     int l, r, t, id; //t表示此前已经进行了t次修改操作
11     bool operator < (const Ask &e2) const { //排序方式也有所变化
12         if ((l - 1) / siz != (e2.l - 1) / siz) return l < e2.l;
13         if ((r - 1) / siz != (e2.r - 1) / siz) return r < e2.r;
14         return t < e2.t;
15     }
16 }ask[N];
17
18 struct Update {
19     int pos, val;
20 }update[N]; //记录第 uid 次修改操作
21
22 int qid, uid;
23
24 int cnt[N];
25 int ans[N];
26 int sum;
27
28 void add(int x) {
29     if (++cnt[x] == 1) {
30         sum++;
31     }
32 }
33
34 void del(int x) {
35     if (--cnt[x] == 0) {
36         sum--;
37     }
38 }
39
40 void mov(int l, int r, int t) { //修改/还原操作
41     auto &[pos, val] = update[t];
42     if (l <= pos && r >= pos) { //如果当前窗口包含第t次操作的位置,进行修改
43         del(a[pos]);
44         add(val);
```

```

45     }
46     std::swap(val, a[pos]);
47 }
48
49 void init() {
50     siz = std::pow(n, 2.0 / 3); // 块的大小
51     std::sort(ask + 1, ask + qid + 1);
52     for (int i = 1, l = 1, r = 0, t = 0; i <= qid; i++) {
53         while (l > ask[i].l) add(a[--l]);
54         while (r < ask[i].r) add(a[++r]);
55         while (l < ask[i].l) del(a[l++]);
56         while (r > ask[i].r) del(a[r--]);
57         while (t < ask[i].t) mov(ask[i].l, ask[i].r, ++t); // 额外在时间维移动
58         while (t > ask[i].t) mov(ask[i].l, ask[i].r, t--);
59         ans[ask[i].id] = sum;
60     }
61 }
62
63 int main() {
64     std::cin >> n >> q;
65     for (int i = 1; i <= n; i++) {
66         std::cin >> a[i];
67     }
68
69     for (int i = 1; i <= q; i++) {
70         char op; std::cin >> op;
71         if (op == 'Q') {
72             int l, r; std::cin >> l >> r;
73             ++qid;
74             ask[qid] = {l, r, uid, qid};
75         }
76         if (op == 'R') {
77             int pos, val; std::cin >> pos >> val;
78             ++uid;
79             update[uid] = {pos, val};
80         }
81     }
82
83     init();
84
85     for (int i = 1; i <= qid; i++) {
86         std::cout << ans[i] << '\n';
87     }
88 }

```

单点修改，查询区间第k小：详见[值域分块](#)

回滚莫队

有的问题在区间转移时，难以同时维护增加节点或者删除节点时的信息，我们可以只实现一个操作，利用回滚解决另一个操作。

只增回滚莫队

例如求区间众数时，增加一个数容易维护，但删数时难以维护。

[AT_joisc2014 c 歴史の研究 - 洛谷](#)

定义一个数 x 的重要程度为 x 乘上 x 在区间内出现的次数。

给定数组 $a[]$ 和 q 次查询：求区间 $[l, r]$ 内最大的重要度。

时间复杂度 $O(N\sqrt{Q})$

按所在块编号从小到大排序，同一块内按右端点从小到大排序。

依次处理每一块，处理前清空信息。

初始时窗口位于当前块右端，大小为0，即 $[R[block]+1, R[block]]$

对于块内每个询问 $[l, r]$ ，若左右端点在同一区间则暴力枚举。否则先扩展右端点至 r ，备份当前信息，左端点再临时往左括至 l ，得到当前查询的答案，再让答案直接回溯到之前的备份，左端点回滚至

$R[block+1]$ 。

```
1  #include <iostream>
2  #include <cstring>
3  #include <vector>
4  #include <algorithm>
5  #include <cmath>
6
7  const int N = 100005;
8  int n, q, siz;
9  int a[N];
10 std::vector<int> hs(1, -2e9);
11
12 struct Ask{
13     int l, r, id;
14     bool operator < (const Ask &e2) const {
15         if ((l - 1) / siz != (e2.l - 1) / siz) return l < e2.l;
16         return r < e2.r; //块内按 r 递增排序
17     }
18 }ak[N];
19
20 int R[N], tot;
21
22 long long cnt[N], ans[N], sum;
23
24 long long brute_force(int l, int r) {
25     long long now = 0;
26     for (int i = l; i <= r; i++) {
```



```

27     cnt[a[i]]++;
28     now = std::max(now, cnt[a[i]] * hs[a[i]]);
29 }
30 for (int i = l; i <= r; i++) { //还原
31     cnt[a[i]]--;
32 }
33 return now;
34 }
35
36 void add(int x) { //增加时同时维护区间信息及答案
37     cnt[x]++;
38     sum = std::max(sum, cnt[x] * hs[x]);
39 }
40
41 void del(int x) { //删除时只维护区间信息,不维护答案
42     cnt[x]--;
43 }
44
45 void init() {
46     siz = std::sqrt(n);
47     std::sort(ak + 1, ak + q + 1);
48
49     tot = (n + siz - 1) / siz;
50     for (int i = 1; i <= tot; i++) { R[i] = std::min(i * siz, n); }
51
52     for (int block = 1, i = 1; block <= tot; block++) { //按块枚举
53         sum = 0;
54         std::memset(cnt, 0, sizeof cnt); //每次切换块时清空区间信息及答案
55         //窗口初始在当前块的右边
56         for (int l = R[block] + 1, r = l - 1; i <= q && (ak[i].l - 1) / siz + 1
== block; i++) {
57             if (ak[i].r <= R[block]) { //诺查询区间在同一块则暴力枚举
58                 ans[ak[i].id] = brute_force(ak[i].l, ak[i].r);
59             }
60             else {
61                 while (r < ak[i].r) add(a[++r]); //扩展右端点
62                 long long bak = sum; //备份当前答案,如果有其它与答案相关的信息也要备份
63                 while (l > ak[i].l) add(a[--l]); //临时扩展左端点
64                 ans[ak[i].id] = sum; //得到答案
65                 sum = bak; //答案直接利用备份回溯
66                 while (l <= R[block]) del(a[l++]); //左端点回滚至R[block]+1
67             }
68         }
69     }
70 }
71
72 int main() {
73     std::cin >> n >> q;
74     for (int i = 1; i <= n; i++) {
75         std::cin >> a[i];
76         hs.emplace_back(a[i]);
77     }

```

```

78
79     for (int i = 1; i <= q; i++) {
80         int l, r; std::cin >> l >> r;
81         ak[i] = {l, r, i};
82     }
83
84     std::sort(hs.begin() + 1, hs.end());
85     hs.erase(std::unique(hs.begin() + 1, hs.end()), hs.end());
86     for (int i = 1; i <= n; i++) {
87         a[i] = std::lower_bound(hs.begin() + 1, hs.end(), a[i]) - hs.begin();
88     }
89
90     init();
91
92     for (int i = 1; i <= q; i++) {
93         std::cout << ans[i] << '\n';
94     }
95 }

```

[P5906 【模板】回滚莫队&不删除莫队 - 洛谷](#)

给定序列 $a[N]$ ， Q 次询问：区间 $[l, r]$ 内，相同的数中的最远间隔距离。

```

1  #include <iostream>
2  #include <cstring>
3  #include <cmath>
4  #include <algorithm>
5  #include <vector>
6
7  const int N = 200005;
8  int n, q, siz, tot, R[N];
9  int a[N];
10 std::vector<int> hs(1, -2e9);
11
12 struct Ask {
13     int l, r, id;
14     bool operator < (const Ask &e2) const {
15         if ((l - 1) / siz != (e2.l - 1) / siz) return l < e2.l;
16         return r < e2.r;
17     }
18 }ak[N];
19
20 int first[N], last[N], sum, ans[N];
21
22 int brute_force(int l, int r) {
23     int now = 0;
24     for (int i = l; i <= r; i++) {
25         if (first[a[i]] == 0) {
26             first[a[i]] = i;
27

```

```

28         else {
29             now = std::max(now, i - first[a[i]]);
30         }
31     }
32     for (int i = l; i <= r; i++) {
33         first[a[i]] = 0;
34     }
35     return now;
36 }
37
38 void addRight(int idx) {
39     int x = a[idx];
40     if (first[x] == 0) first[x] = idx;
41     last[x] = idx;
42     sum = std::max(sum, last[x] - first[x]);
43 }
44
45 void addLeft(int idx) {
46     int x = a[idx];
47     if (last[x] == 0) last[x] = idx;
48     else sum = std::max(sum, last[x] - idx);
49 }
50
51 void delLeft(int idx) {
52     int x = a[idx];
53     if (last[x] == idx) last[x] = 0;
54 }
55
56 void init() {
57     siz = std::sqrt(n);
58     std::sort(ak + 1, ak + q + 1);
59
60     tot = (n + siz - 1) / siz;
61     for (int i = 1; i <= tot; i++) R[i] = std::min(i * siz, n);
62
63     for (int block = 1, i = 1; block <= tot; block++) {
64         sum = 0;
65         std::memset(first, 0, sizeof first);
66         std::memset(last, 0, sizeof last);
67         for (int l = R[block] + 1, r = l - 1; i <= q && (ak[i].l - 1) / siz + 1
68 == block; i++) {
69             if (ak[i].r <= R[block]) {
70                 ans[ak[i].id] = brute_force(ak[i].l, ak[i].r);
71             }
72             else {
73                 while (r < ak[i].r) addRight(++r);
74                 long long bak = sum;
75                 while (l > ak[i].l) addLeft(--l);
76                 ans[ak[i].id] = sum;
77                 sum = bak;
78                 while (l <= R[block]) delLeft(l++);

```

```

79     }
80 }
81 }
82
83 int main() {
84     std::ios::sync_with_stdio(false); std::cin.tie(0);
85     std::cin >> n;
86     for (int i = 1; i <= n; i++) {
87         std::cin >> a[i];
88         hs.emplace_back(a[i]);
89     }
90
91     std::cin >> q;
92     for (int i = 1; i <= q; i++) {
93         int l, r; std::cin >> l >> r;
94         ak[i] = {l, r, i};
95     }
96
97     std::sort(hs.begin() + 1, hs.end());
98     hs.erase(std::unique(hs.begin() + 1, hs.end()), hs.end());
99     for (int i = 1; i <= n; i++) {
100         a[i] = std::lower_bound(hs.begin() + 1, hs.end(), a[i]) - hs.begin();
101     }
102
103     init();
104
105     for (int i = 1; i <= q; i++) {
106         std::cout << ans[i] << '\n';
107     }
108 }

```

只删回滚莫队

例如求区间mex时，删除一个数容易维护，但增加一个数难以维护

[P4137 Rmq Problem / mex - 洛谷](#)

给定序列 $a[N]$ ， Q 次询问：求区间 $[l, r]$ 的 mex

时间复杂度 $O(N\sqrt{Q})$

按所在块编号从小到大排序，同一块内按右端点从大到小排序。

依次处理每一块，初始时窗口对齐至 $[L[block], n]$ ，备份当前区间答案 $last$ 。

对于块内每个询问 $[l, r]$ ，右端点缩短至 r ，备份当前答案 bak ，左端点再临时缩短至 l ，得到前询问答案，再让答案回溯到之前的备份 bak ，左端点回滚至 $L[block]$ 。

每块处理完后，让右端点回滚至 n ，答案利用备份 $last$ 回溯。

```

1  #include <iostream>
2  #include <cmath>
3  #include <algorithm>
4  #include <cstring>
5
6  const int N = 200005;
7  int n, q, siz, tot, L[N];
8  int a[N];
9
10 struct Ask {
11     int l, r, id;
12     bool operator < (const Ask &e2) const {
13         if ((l - 1) / siz != (e2.l - 1) / siz) return l < e2.l;
14         return r > e2.r; //块内按右端点从大到小排序
15     }
16 }ak[N];
17
18 int mex, ans[N], cnt[N];
19
20 void add(int x) { //加点时只维护区间信息
21     cnt[x]++;
22 }
23
24 void del(int x) { //删点时维护区间信息及答案
25     if (--cnt[x] == 0) {
26         mex = std::min(mex, x);
27     }
28 }
29
30 void init() {
31     siz = std::sqrt(n);
32     std::sort(ak + 1, ak + q + 1);
33
34     tot = (n + siz - 1) / siz;
35     for (int i = 1; i <= tot; i++) L[i] = (i - 1) * siz + 1; //记录每块的左端点
36
37     int l = 1, r = n;
38     for (int i = 1; i <= n; i++) cnt[a[i]]++; //初始窗口为[1,n]
39     while (cnt[mex]) mex++;
40     for (int block = 1, i = 1; block <= tot; block++) {
41         while (l < L[block]) del(a[l++]); //左端点对齐,得到区间[L[block],n]信息
42         int last = mex; //备份当前区间信息
43         for (; i <= q && (ak[i].l - 1) / siz + 1 == block; i++) {
44             while (r > ak[i].r) del(a[r--]); //右端点缩短
45             int bak = mex; //备份当前答案
46             while (l < ak[i].l) del(a[l++]); //左端点临时缩短
47             ans[ak[i].id] = mex; //得到当前查询答案
48             mex = bak; //回溯
49             while (l > L[block]) add(a[--l]); //左端点回滚至L[block]
50         }
51         while (r < n) add(a[++r]); //右端点回滚至n

```

```

52     mex = last; //mex回溯为[L[block-1],n],在下一块通过左端点对齐得到[L[block+1],n]
    的信息及答案
53     }
54 }
55
56 int main() {
57     std::cin >> n >> q;
58     for (int i = 1; i <= n; i++) {
59         std::cin >> a[i];
60     }
61
62     for (int i = 1; i <= q; i++) {
63         int l, r; std::cin >> l >> r;
64         ak[i] = {l, r, i};
65     }
66
67     init();
68
69     for (int i = 1; i <= q; i++) {
70         std::cout << ans[i] << '\n';
71     }
72 }

```

随机化

[例题：P7261 [COCI 2009/2010 #3](#)] [PATULJCI](#) - 洛谷

给定一个序列 $a[N]$ ， Q 次询问：求区间 $[l, r]$ 中，出现次数超过一半的数

若一个数 c 在区间内出现的次数超过了一半，那么我们在该区间随机 t 次，每次随机到一个数不是 c 的概率小于 $(\frac{1}{2})^t$ ，随着 t 增大，几乎必定能够遇到 c 。

对于每个查询，我们在区间内随机 t 次，每次用二分判断区间内当前数出现的次数即可。

时间复杂度 $O(QT \log N)$ ，正确率约为 $1 - Q(\frac{1}{2})^T$ 。其中 T 为随机化次数，小了容易WA，大了容易TLE。

```

1  #include <bits/stdc++.h>
2
3  const int N = 300005;
4  int n, c, q;
5  int a[N], ans[N];
6  std::vector<int> v[N];
7

```

```

8  auto SEED = std::chrono::steady_clock::now().time_since_epoch().count();
9  std::mt19937_64 rng(SEED);
10
11 int main() {
12     std::cin >> n >> c;
13     for (int i = 1; i <= n; i++) {
14         std::cin >> a[i];
15         v[a[i]].emplace_back(i);
16     }
17
18     std::cin >> q;
19     int lim = 20;
20     for (int w = 1; w <= q; w++) {
21         int l, r; std::cin >> l >> r;
22         if (r - l + 1 <= lim) { //小区间直接枚举
23             std::map<int, int> cnt;
24             for (int i = l; i <= r; i++) {
25                 cnt[a[i]]++;
26                 if (cnt[a[i]] > (r - l + 1) / 2) {
27                     ans[w] = a[i];
28                     break;
29                 }
30             }
31         }
32         else {
33             std::set<int> se;
34             for (int i = 0; i < lim; i++) {
35                 int id = rng() % (r - l + 1);
36                 se.insert(a[id + l]); //在区间[l, r]内随机lim个数
37             }
38
39             for (auto &x : se) {
40                 auto k1 = std::lower_bound(v[x].begin(), v[x].end(), l);
41                 auto k2 = std::upper_bound(v[x].begin(), v[x].end(), r);
42                 if (k2 - k1 > (r - l + 1) / 2) { //二分判断区间[l, r]内x的出现次数
43                     ans[w] = x;
44                     break;
45                 }
46             }
47         }
48     }
49
50     for (int i = 1; i <= q; i++) {
51         if (ans[i]) std::cout << "yes " << ans[i] << '\n';
52         else std::cout << "no\n";
53     }
54 }

```